UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET
Katedra za računarstvo

DETEKCIJA POZE TELA NA OSNOVU VIDEA

- DIPLOMSKI RAD -

Zadatak:

Upoznati se sa tehnikama dubokog učenja za detekciju poze tela na osnovu videa. Istražiti postojeće modele koji se koriste za rešavanje ovog problema. Na osnovu prethodno utreniranog modela razviti aplikaciju koja pomaže u vežbanju detektujući ispravnost obavljanja vežbe.

Mentor: prof. dr Aleksandar Milosavljević

Kandidat: Nikola Trajković 16427

Komisija:

1. _____ Datum prijave: _____

2. _____ Datum predaje: _____

3. _____ Datum odbrane: _____

Niš, 2020.

Sadržaj

Uvod ... 3
Detekcija poze tela ... 4
Konvolucione neuronske mreže ... 6
Primeri konvolucionih neuronskih mreža ... 9
Korišćenje konvolucionih neuronskih mreža za detekciju poze ... 13
Tehnike dubokog učenja za detekciju poze tela ... 16
Primena detekcije poze tela ... 19
Postojeći modeli za detekciju poze tela ... 20
Efikasna lokalizacija objekata pomoću konvolucionih neuronskih mreža ... 20
OpenPose ... 23
Aplikacija za asistenciju u vežbanju ... 30
Zaključak ... 42
Reference ... 44

Uvod

Sama ideja nastanka računara predstavlja težnju čoveka da napravi mašinu koja će moći da ga oponaša i zameni u brojnim aktivnostima. Razvoj tehnologije konstantno doprinosi ostvarivanju ovog cilja korak po korak. Svedoci smo infiltracije savremene tehnologije u svaki deo naših života i gotovo je nezamisliv život bez nje. Jedna od ideja u razvoju računara jeste omogućavanje računarima da "vide", odnosno mogućnost mašina da prepoznaju slike. Oblast računarstva koja se bavi ovom problematikom se naziva računarski vid (eng. *computer vision*).

Računarski vid je grana računarske nauke koja se bavi razvijanjem tehnika i algoritama koji omogućavaju kompjuterima da razumeju sadržaj digitalnih slika, bilo da su u pitanju fotografije ili video. Problem nije lako rešiv i predmet je konstantnog rada i istraživanja ove oblasti. Ono što je za ljude veoma lako, prepoznavanje predmeta, okoline, drugih ljudi, za mašinu je veoma težak proces. Međutim poslednja decenija je donela puno novih rešenja i algoritama u ovoj oblasti i praćena velikim razvojem tehnologije i hardvera omogućila rešavanje problema koji su se dugo činili nerešivim.

Svaki čovek u današnje vreme ima pametni mobilni telefon, pa se samim tim već susreo sa nekom od implementacija računarskog vida. Na primer mobilni telefoni nude mogućnost automatskog otključavanja na osnovu detekcije lica. Problemi kao što su detekcija i prepoznavanje lica, automatsko prepoznavanje osoba na slici, klasifikacija objekata na slici i drugi su sada već postali u neku ruku trivijalni i veoma lako se rešavaju korišćenjem konvolucionih neuronskih mreža. Jedan od težih izazova u računarskom vidu je bio kreirati tehnike i načine detekcije poze čoveka na slikama, zbog postojanja nebrojeno poza koje telo može zauzeti u prostoru.

Prvi u rešavanju ovog problema bio je Kinect razvijen od strane firme Microsoft koji je koristio dubinsku sliku kako bi odredio pozu tela na videu u realnom vremenu. Međutim tehnologija se menja ogromnom brzinom i svedoci smo da generalno nove stvari vrlo brzo zastarevaju, tako je bilo i sa Kinect-om gde je pravac istraživanja detekcije poze sa dubinskih slika pomeren na obične RGB slike. Kao i u drugim gore navedenim rešenjima ovakvom pravcu razvoja najviše su doprinele duboke konvolucione neuronske mreže ili duboko učenje. Princip rada dubokih konvolucionih mreža, njihova primena u rešavanju problema detekcije poze tela na slikama i opis postojećih modela koji koriste ovaj pristup biće tema ovog rada.

Rad je organizovan u pet poglavlja. Nakon Uvoda, u drugom poglavlju je obrađen problem detekcije poze tela. Predstavljen je sam pojam i moguće podele detekcije poze, a nakon toga najviše pažnje je posvećeno principu rada dubokih konvolucionih neuronskih mreža i njihovom primenom u detekciji poze oslanjajući se na rad DeepPose iz 2015. godine. Takođe predstavljene su tehnike koje se koriste kod dubokog učenja pri detekciji poze više ljudi na slici. U trećem poglavlju obrađena su dva bitna modela za detekciju poze tela, gde je akcenat stavljen na OpenPose koji detektuje pozu na osnovu videa u realnom vremenu. U narednom poglavlju pod nazivom Aplikacija za asistenciju u vežbanju opisana je aplikacija koja prikazuje mogućnost i princip praćenja poze tela vežbača prilikom izvođenja vežbe. Zadnje poglavlje pod nazivom Zaključak, se osvrće na sva prethodna poglavlja i izvlači kratak siže rada.

Detekcija poze tela

Detekcija poze tela je veoma zanimljiv problem u računarskom vidu i predmet je brojnih istraživanja u proteklih tridesetak godina. Detekcija poze tela je tehnika računarskog vida koja predstavlja problem identifikacije i lokalizacije ključnih tačaka na ljudskom telu na slici ili videu. Ključne tačke na ljudskom telu su najčešće zglobovi kao na primer lakat, rame, koleno, skočni zglob, kuk, ali se mogu odnositi i na manje delove tela kao što su oči, nos, obrve, prsti..

Čovek je veoma fleksibilno biće, i pomeranjem svojih ekstremiteta njegovi zglobovi (ključne tačke) mogu zauzimati različite položaje u prostoru i biti relativno udaljeni jedni od drugih. Kod čvrstih predmeta detekcija je daleko jednostavnija jer su njihove ključne tačke skoro uvek na podjednakoj udaljenosti i njihova detekcija zavisi samo od položaja celog predmeta u prostoru. Pored brojnih varijacija u pozama tela, ono što čini problem detekcije poze većim jesu i mali i jedva vidljivi zglobovi, promena odeće i obuće, česte promene osvetljenja i još dosta faktora.

Kod određivanja poze tela može se izvršiti podela i napraviti jasna razlika između:

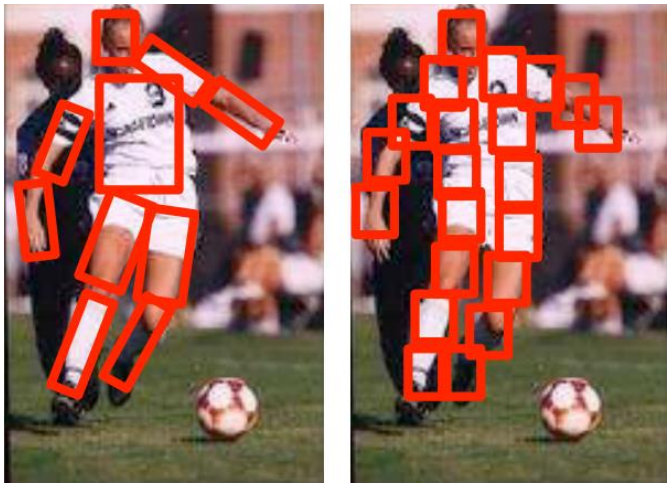
- **2D procene poze** i
- **3D procene poze tela**.

Kada govorimo o 2D proceni poze, očekujemo kao izraz dvodimenzionalne koordinate (x, y) za svaku ključnu tačku u relativnom 2D prostoru slike. Kod 3D procene poze radi se o procenjivanju i dodavanju treće dimenzije *z* (dubine) dvodimenzionalnim koordinatama ključnih tačaka. Kao izlaz dobijaju se trodimenzionalne koordinate (x, y, z) na osnovu kojih je moguće izvršiti predviđanje prostornog pozicioniranja osobe. 3D procena poze je daleko kompleksniji problem i neće biti tema ovog rada, akcenat će biti na metodama i tehnikama 2D procene poze tela.

Problem detekcije poze može se rešiti korišćenjem dubinskih kamera, odnosno kamera koje pored RGB slike, snimaju i dubinsku sliku. To se postiže time što pored RGB kamere, postoji i projektor koji projektuje infracrvene zrake u prostor, a infracrvena kamera ih snima. Na osnovu tih informacija kreira se dubinska slika koja pored boje piksela ima i informaciju o udaljenosti tog piksela od kamera. Na ovaj način lako se detektuje čovek na slici, a problem detekcije njegove poze predstavlja zapravo problem klasifikacije. Za svaki piksel se izvrši njegova klasifikacija kao neki deo tela i primenom *Mean shift* algoritma izvrši procena položaja zglobova. Međutim novija istraživanja su pokazala da zapravo nije neophodno snimati dubinsku sliku kako bi se odredila poza tela, već je to moguće odrediti na osnovu RGB slike i to će biti tema u ovom radu.

Važno je napomenuti i razliku između detekcije **jedne osobe** i **više osoba** na slici, gde postoje različiti pristupi u rešavanju ovih problema. **Klasični pristup** se bazira na predstavljanju objekta kao kolekcije poređanih delova u deformabilnoj konfiguraciji. Deo predstavlja neki obrazac (eng. *template*) koji se pronalazi na slici i kasnije povezuje sa svojim susednim delovima. Svaki par delova ima i svoja geometrijska ograničenja. Kada su delovi parametrizovani lokacijom piksela i orijentacijom, rezultujuća struktura može modelovati

određenu artikulaciju. [1] Autori Yang i Ramanan [16] proširuju klasičan pristup uvodeći takozvani fleksibilan model mešovitih delova (eng. *flexible mixture-of-parts model*), gde se jedan deo kod klasičnog pristupa prikazuje kao veza manjih delova (slika 1). Na ovaj način se postiže bolja detekcija jer se posmatraju i lokalne promene pojedinačnih delova. Dakle, generalna ideja je prvo detektovati pojedinačne delove tela, pa zatim vršiti njihovo povezivanje i kreiranje poze tela. Ovakav metod radi samo sa slikom jednog čoveka, pa ga to čini skoro neupotrebljivim u većini realnih situacija gde se na slici može naći nekoliko ljudi. [2]



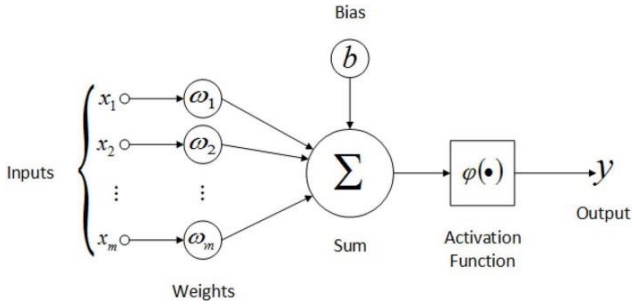
Slika 1: Klasični model (levo) i fleksibilni model mešovitih delova (desno) [16]

Rad Toshev-a i Szegedy-a iz Googla pod nazivom *DeepPose* [17] prikazanom na *CVPR* konferenciji (*Conference on Computer Vision and Patern Recognition*) 2014. godine predstavio je metod detekcije ljudske poze zasnovan na korišćenju dubokih neuronskih mreža (eng. *Deep Neural Network - DNN*). Prema autorima [17] problem detekcije poze je formulisan kao regresioni problem ljudskih tačaka zasnovan na DNN i predstavili su kaskadu takvih DNN regresora koja je rezultirala visokom preciznošću u samoj proceni. Nakon ovog rada klasični pristup je zamenjen pristupom zasnovanom na **dubokom učenju**, odnosno korišćenju dubokih konvolucionih neuronskih mreža (eng. *convolutional neural network* - **CNN**) i pokazao se veoma uspešnim u ovoj oblasti. U nastavku rada sledi opis i prikaz arhitektura konvolucionih neuronskih mreža kao i njihova adaptacija za rešavanje problema detekcije poze.

Konvolucione neuronske mreže

Konvoluciona neuronska mreža (*CNN* ili *ConvNets*) je višeslojna duboka neuronska mreža sa specijalnom arhitekturom da detektuje kompleksne fičere (eng. *features*) u podacima (slikama). CNN imaju široku upotrebu u različitim oblastim računarskog vida kao na primer detekcije objekata ili ljudi na slici, oponašanje čula vida kod robota, autopilota kod automobila. Kao i kod veštačkih neuronskih mreža, konvolucione neuronske mreže su inspirisane radom ljudskog mozga. CNN su sposobne da klasifikuju sliku na osnovu detektovanih karakteristika (fičera) slično kao što ljudski mozak detektuje fičere kako bi identifikovao objekte. [5]

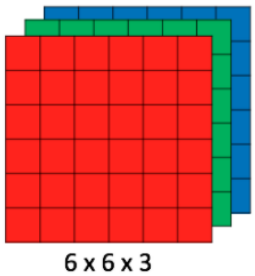
Kao i kod veštačke neuronske mreže, konvoluciona neuronska mreža se sastoji iz veštačkih neurona koji imaju određeni broj ulaza, gde je za svaki ulaz vezana određena težina, prag, svoju aktivacionu funkciju i izlaz. Veštački neuron je zapravo osnovni procesni element svake neuronske mreže koji obavlja neku relativno jednostavnu matematičku funkciju nad ulazima i njihovim težinama.



Slika 2: Model veštačkog neurona [6]

Veštački neuroni su grupisani u slojeve koji su međusobno povezani i na taj način kreiraju slojevitu neuronsku mrežu. Mreža se najčešće sastoji od jednog ulaznog sloja, više sakrivenih slojeva i jednog izlaznog sloja. Neuroni se mogu podeliti na osnovu toga kom sloju pripadaju na ulazne, izlazne i sakrivene. Obično izlazi neurona u izlaznom sloju predstavljaju i sam izlaz iz mreže.

Ono što razlikuje neuronsku mrežu i konvolucionu neuronsku mrežu jeste sam ulaz, kod neuronske mreže ulaz je vektor podataka, dok je kod konvolucione ulaz slika. Ukoliko govorimo o RGB slici, ona je predstavljena kao skup od 3 matrice gde svaka matrica predstavlja određeni kanal za svaku od boja (crvenu, zelenu, plavu). Svaki element u matrici uzima vrednost od 0 do 255 i predstavlja količinu određene boje koja učestvuje u formiranju piksela na slici.



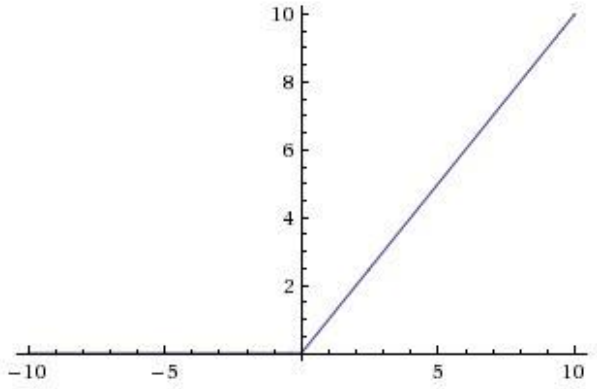
Slika 3: Predstavljanje slike preko RGB matrica [7]

Kako bi bio omogućen rad sa slikama, konvoluciona neuronska mreža sadrži jedan ili više konvolucionih slojeva. Konvolucioni sloj se može posmatrati kao ekstraktor fičera iz ulazne slike. Pored **konvolucionih slojeva**, CNN sadrži **ReLU** (*Rectified Linear Unit*) slojeve, **puling** (pooling) slojeve i **potpuno povezani** sloj.

Kod **konvolucionog sloja** se nad ulaznom slikom primenjuje detektor fičera ili filtar (kernel). Filtar je matrica najčešćih dimenzija 3x3, 5x5 ili 7x7 koja se množi sa matricama ulazne slike po metodi klizajućeg prozora i kao rezultat se dobija matrica manjih dimenzija – fičer mapa. Cilj ovog koraka je da redukuje veličinu slike i učini obradu bržom i lakšom. Tokom ovog procesa dosta fičera se izgubi, ali su bitni fičeri zadržani. To su fičeri koji su jedinstveni za identifikaciju specifičnih objekata. Kako bi se sprečilo gubljenje informacija (fičera) sa slika kreira se više fičer mapa. Svaka mapa detektuje lokacije određenih fičera na slici. [5]

Sledeći korak je primena **ReLU** aktivacione funkcije kako bi se povećala nelinearnost u CNN. ReLU funkcija je najkorišćenija aktivaciona funkcija u modelima dubokog učenja. Funkcija vraća 0 ukoliko primi negativnu vrednost, a za pozitivnu vrednost *x,* vraća tu vrednost nazad.

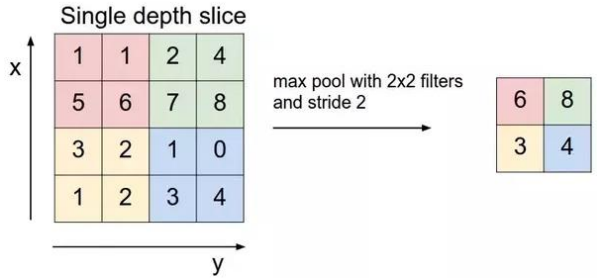
$$f(x)=\max(0,x)$$



Slika 4: Grafički prikaz ReLU aktivacione funkcije

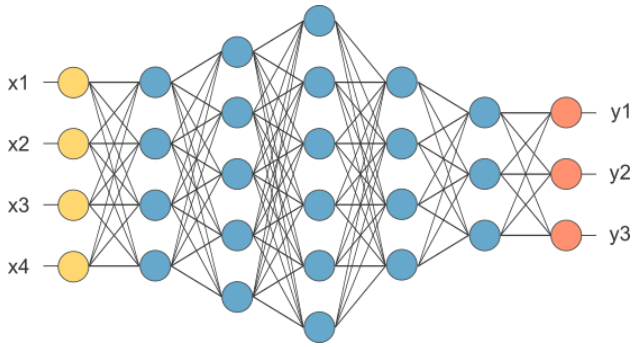
Slike se sastoje iz različitih objekata koji nisu međusobno linearni. Bez primene ove funkcije klasifikacija u slikama bi bila tretirana kao linearan problem, dok je on zapravo nelinearan. [5]

Prostorna invarijantnost je koncept gde lokacija objekta na slici ne utiče na sposobnost neuronske mreže da detektuje njegove specifične fičere. **Puling** omogućava da CNN detektuje fičere na različitim slikama nezavisno od razlike u osvetljenju i različitih uglova slika. Postoje različiti tipovi pulinga, kao na primer, maksimalni i minimalni puling (eng. *max-pooling, min-pooling*). Puling se primenjuje u puling sloju i sličan je primeni filtra u konvoluciji. Neki od pulinga se primenjuje na svaku od dobijenih matrica fičera zasebno kako bi se dobile nove pulovane fičer mape (matrice). Dimenzija puling filtra mora biti manja od veličine matrice fičera i uglavnom je 2x2, sa pomerajem 2, što za rezultat ima smanjenje veličine fičer mape za četvrtinu. Puling filtar se kreće po metodi klizajućeg prozora s leva na desno preko cele fičer mape i primenjuje svoju operaciju, npr. maksimalni puling bira najveću vrednost unutar prozora. Najčešće korišćene puling operacije su prosečni puling i maksimalni puling. Maksimalni puling, na primer, se koristi kako bi sačuvao glavne fičere dok ujedno redukuje veličinu slike. [5]



Slika 5: Maksimalni puling [7]

Nakon kreiranja pulovane fičer mape sledeći korak je njena transformacija u vektor koji se zatim prosleđuje **potpuno povezanom sloju** (slika 6) na procesiranje. Vektor se dobija konkateniranjem redova matrica i relativno je lak proces. Ovako dobijeni vektor fičera je sada dosta manjih dimenzija od veličine ulaznih matrica za svaki od 3 kanala, pa je time omogućena lakša i brža obrada u realnom vremenu. Potpuno povezani sloj je na ulazu povezan sa poslednjim konvolucionim/puling slojem i zadužen je za klasifikaciju prethodno pripremljenih fičera. Svaki neuron u izlaznom sloju zapravo predstavlja jednu klasu, a njegov izlaz verovatnoću da se ta klasa nalazi na slici.



Slika 6: Primer potpuno povezanog sloja [7]

Poslednji sloj je najčešće *softmax* sloj koji kao aktivacionu funkciju koristi *softmax* funkciju, odnosno normalizovanu eksponencijalnu funkciju. Ova funkcija normalizuje verovatnoće predviđenih klasa tako da njihov zbir bude 1.

$$\sigma : \mathbb{R}^K \to (0,1)^K$$

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}} \quad \text{for } j = 1, \ldots, K.$$

Slika 7: Softmax funkcija [8]

Najčešća arhitektura konvolucionih neuronskih mreža obuhvata stek od nekoliko konvolucioninih-ReLU slojeva nakon čega sledi puling sloj i ovaj patern se ponavlja sve dok se ulazna slika prostorno ne redukuje na manju veličinu kada se prosleđuje potpuno povezanom sloju na klasifikaciju. Najčešća arhitektura se može opisati sledećim obrascem:

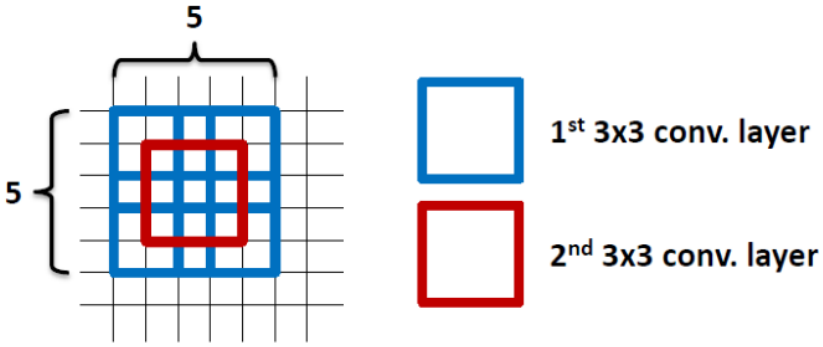
```
INPUT -> [[CONV -> RELU]*N -> POOL?]*M -> [FC -> RELU]*K -> FC,
```

gde * predstavlja ponavljanje, a POOL? opcioni puling sloj. [9]

Primeri konvolucionih neuronskih mreža

VGGNet je konvoluciona neuronska mreža razvijena od strane *VGG-a* (eng. *Visual Geometry Group*) sa Oksford univerziteta. Zauzela je drugo mesto na ILSVRC 2014 (*ImageNet Large Scale Visual Recognition Challange*) takmičenju sa greškom u klasifikaciji od 7.3%, dok je pobednik bio GoogLeNet sa 6,7% greškom. Međutim što se tiče zadatka lokalizacije, VGGNet se pokazao boljim. Takođe to je bila godina kada su modeli za duboko učenje smanjili grešku u klasifikaciji na ispod 10%. Ova mreža je jako bitna jer je bila osnova za razvoj drugih modela koji su nadogradili VGGNet ili su bazirani na ideji VGGNet-a o korišćenju filtra veličine 3x3, kao na primer VGG-11, VGG-11 (LRN), VGG-13, VGG-16 i VGG-19.

Mreže koje su pobeđivale na ILSVRC takmičenjima 2012. (AlexNet) i 2013. (ZFNet), bazirale su se na korišćenju filtara veće veličine 11x11 i 7x7, respektivno. Ideja kod VGGNet-a bila je kaskadno korišćenje manjeg filtra 3x3 čime se dobija isti efekat kao primena većih filtara, ali se dobija na performansama i redukciji parametara mreže. Na primer, korišćenje dva konvoluciona sloja sa filtrom 3x3, zapravo se pokriva prostor od 5x5 kao što se ilustrovano na slici 7.

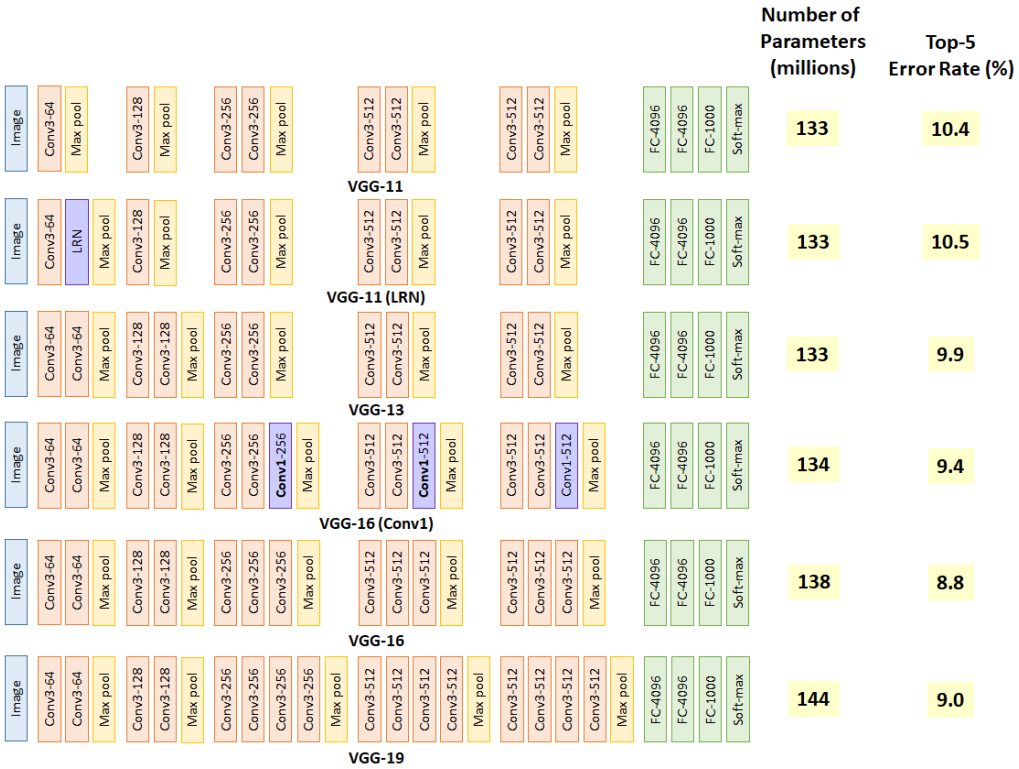


Slika 8: Primer korišćenja dva sloja 3x3 filtra umesto jedne veličine 5x5 [13]

Takođe, korišćenje 3 sloja sa filtrom 3x3 je identično jednoj primeni filtra veličine 7x7 i tako dalje. Smanjivanje broja parametara mreže je takođe jedan od benefita korišćenja manjeg filtra. Na primer, pod pretpostavkom da postoji jedan filtar po sloju, jedan sloj na ulazi i izuzimajući *bias* neurone dobijaju se sledeći rezultati:

- za jedan sloj 5x5 filtra broj parametara je 25 (5*5), dok je za 2 sloja 3x3 filtara taj broj 18 (3*3 + 3*3). U ovom slučaju broj parametara je redukovan za 28%.
- za jedan sloj 7x7 filtra broj parametara iznosi 49 (7*7), a za tri sloja 3x3 filtara broj parametara je 27 (3*3*3). U ovom slučaju broj parametara je redukovan za 45%.
- kod jednog sloja 11x11 filtra broj parametara je 121 (11*11), a za 5 slojeva 3x3 filtara taj broj iznosi 45 (3*3*5), gde je redukcija parametara čak 63%. [13]

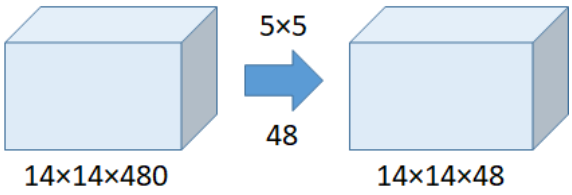
Još jedna stvar koju je potrebno naglasiti kod VGGNet-a jeste ta da se broj filtara duplira posle svakog maksimalnog puling sloja, a ideja je da se na taj način smanjuju prostorne dimenzije, a povećava dubina. Na slici 8, mogu se videti različite arhitekture mreža koje se baziraju na VGGNet-u. Potrebno je primetiti da se u svakom sloju vrši konvolucija sa filtrom 3x3 (*Conv3*) i da se broj filtara u svakom sloju duplira (64 – 128 - 256 ...).



Slika 9: Primeri različitih VGG arhitektura [13]

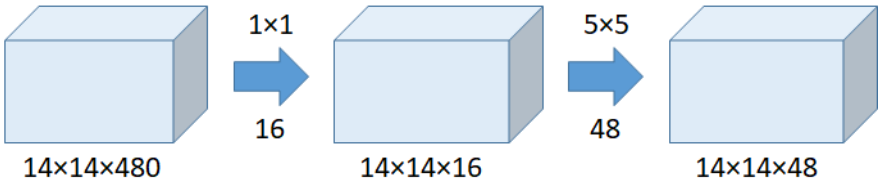
GoogLeNet je konvoluciona neuronska mreža razvijena od strane *Google*-a i bila je pobednik ILSVRC takmičenja 2014. godine sa greškom u klasifikaciji od 6,7%. Pristup i arhitektura kod ove mreže je potpuno drugačiji od VGG, ZFNet i AlexNet. Ova mreža koristi 1x1 konvoluciju i globalni prosečni puling (eng. *global avarage pooling*) na samom kraju mreže umesto potpuno povezanog sloja. Takođe, još jedna novina koja se ovde koristi jeste početni modul (eng. *inception module*) i ima ih ukupno 9 u arhitekturi mreže. Sama mreža se sastoji iz 22 sloja.

Korišćenje konvolucije 1x1 (ili konvolucije uskog grla) omogućava redukciju veličine modela (dubinu) čime se postize redukcija u izračunavanju. Takođe 1x1 konvolucija pomaže i u rešavanju problema pretreniranosti mreže. Primer iz [14] ilustruje benefite ovakvog pristupa. U primeru se prikazuje korišćenje konvolucije 5x5 bez primene 1x1 konvolucije koja zahteva 112,9 miliona operacija (14*14*48*5*5*480), slika 9.



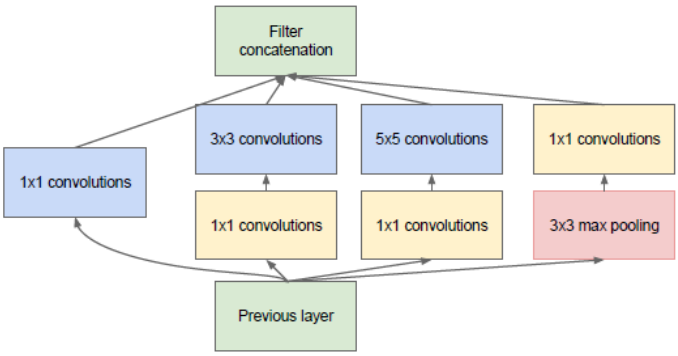
Slika 10: 5x5 konvolucija bez korišćenja 1x1 konvolucije [14]

Ukoliko se iskoristi 1x1 konvolucija dolazi se do drastičnog smanjenja broja operacija. Konvolucija 1x1 zahteva 1,5 milion operacija (14*14*16*1*1*480). Nakon nje sledi 5x5 konvolucija nad podacima čija je dubina drastično smanjena, pa sada ona zahteva 3,8 miliona operacija. Ukupno to obuhvata 1,5M + 3,8M = 5,3 miliona operacija što je dosta manje od 112,9 miliona. Slika 10 prikazuje postupak korišćenja konvolucije uskog grla. [14]



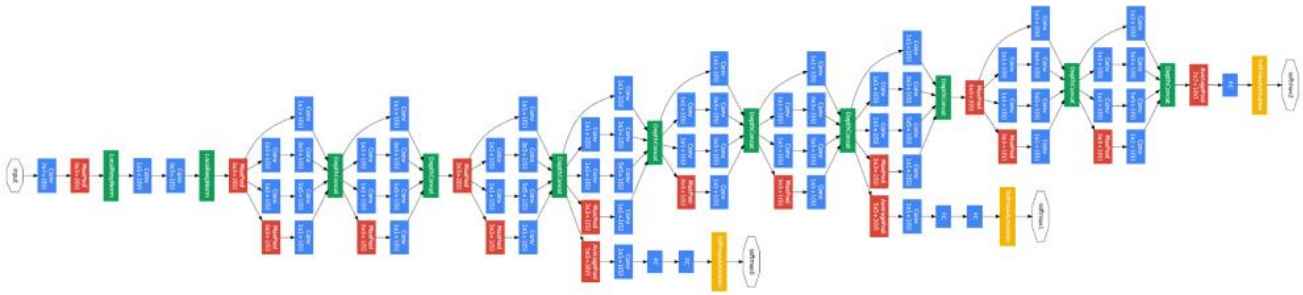
Slika 11: Korišćenje 1x1 pre primene 5x5 konvolucije [14]

Inception modul, prikazan na slici 12, obuhvata korišćenje 1x1, 3x3, 5x5 konvolucija i 3x3 maksimalnog pulinga nad izlazom prethodnog sloja i spajanje njihovih rezultata kao izlaz iz modula. Ideja je da se nad ulaznom slikom primene konvolucije različitih veličina i maksimalni puling kako bi se izdvojili različiti tipovi fičera i njihovom kombinacijom dobio izlaz. Kako bi se redukovao broj operacija ovaj modul obuhvata i dodatne 1x1 konvolucije.



Slika 12: Inception modul [14]

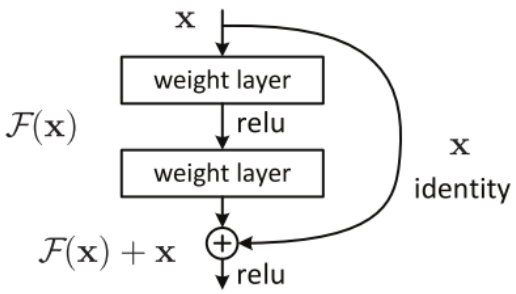
Na samom kraju mreže nalazi se globalni prosečni puling sloj koji je iskorišćen umesto potpuno povezanog sloja kako bi se smanjio broj parametara mreže. Na primeru *AlexNet* mreže može se uočiti da potpuno povezani sloj sadrži oko 90% svih parametara. Korišćenje velike širine i dubine mreže omogućava GoogLeNet-u da ukloni potpuno povezani sloj bez uticaja na tačnost. Na slici 13 prikazana je potpuna arhitektura GoogLeNet-a.



Slika 13: Arhitektura GoogLeNet mreže [14]

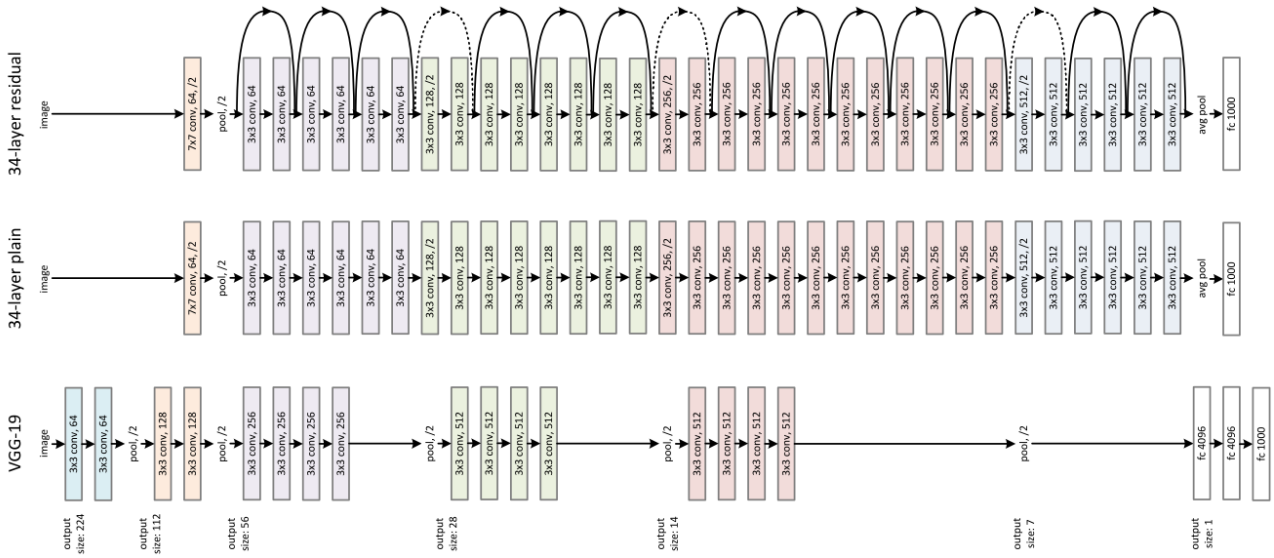
ResNet je konvoluciona neuronska mreža koja je pobedila 2015. na ILSVRC takmičenju sa top-5 greškom od 3,6%. Poređenja radi, greška kod čoveka je oko 5%, što znači da ova mreža ima bolje rezultate i od ljudi. Sama mreža je inspirisana idejom VGGNet-a, ali uvodi novi rezidualni pristup pri obučavanju mreže kako bi se olakšalo njeno treniranje. Takođe, ovaj pristup omogućio je kreiranje još dubljih neuronskih mreža tako da je pobednička verzija imala 152 sloja. To je skoro 7 puta više od GoogLeNet-a koji je bio pobednik godinu dana ranije. Kod ResNet-a se ne koristi maksimalni puling, nema skrivenih potpuno povezanih slojeva, a ono što je preuzeto od VGGNet-a pored dizajna jeste i korišćenje 3x3 filtra u svim slojevima izuzev prvog gde je filtar veličine 7x7.

Glavna ideja rezidualnog učenja jeste uvođenje takozvane veze prečice identiteta (eng. *identity shortcut connection*) koja preskače jedan ili više konvolucionih slojeva kao što je prikazano na slici 14. [15]



Slika 14: Veza prečice identiteta [15]

Ovom "prečicom" se postiže da se ulazna slika propagira kroz mrežu, odnosno izlaz jednog sloja propagira u nastavku mreže. To se postiže na taj način što se originalni ulaz (identitet) *x* sabira sa rezultatom sloja/slojeva *F(x)* i prosleđuje sledećem sloju. Na slici 15 prikazana je arhitektura ResNet mreže sa 34 sloja sa korišćenjem rezidualnog pristupa i na kojoj se mogu uočiti sličnosti sa VGG principom arhitekture.



Slika 15: Primer ResNet arhitekture sa 34 sloja sa (gore) i bez (sredina) rezidualnog pristupa i poređenje sa VGG-19 (dole) [15]

Korišćenje konvolucionih neuronskih mreža za detekciju poze

U prethodnom podnaslovu prikazano je nekoliko važnih konvolucionih mreža koje su se veoma dobro pokazale u rešavanju problema klasifikacije. Problem lokalizacije, tačnije lokalizacije ključnih tačaka na čovekovom telu i detektovanje poze, je takođe rešen primenom određenih modifikacija konvolucionih neuronskih mreža. U nastavku sledi opis rada DeepPose koji koristi ovakav pristup.

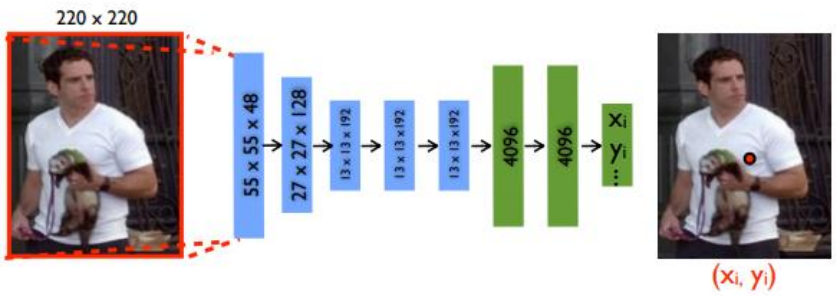
DeepPose ili procena ljudske poze preko dubokih neuronskih mreža je prvi rad koji je koristio duboko učenje za rešavanje problema detekcije poze. Kao što je u jednom od podnaslova navedeno, autori ovog rada [17] problem detekcije poze su formulisali kao regresioni problem ljudskih tačaka zasnovan na DNN i predstavili kaskadu takvih DNN regresora koja je rezultirala visokom preciznošću u proceni. Autori uvode i novi pristup o rasuđivanju o pozi na holistički način u kome iako su neki zglobovi sakriveni oni se mogu proceniti ako se poza posmatra u celovitosti. Takođe rad govori o tome da CNN prirodno obezbeđuju ovakav način rasuđivanja. [1]

Lokacija svakog ljudskog zgloba se određuje korišćenjem regresora koji koriste cele slike kao ulaz i 7-slojnu generičku konvolucionu mrežu. Ovakav pristup ima sledeće dve prednosti:

- CNN na ovaj način sagledava kontekst celokupnog tela kako bi odredila poziciju zgloba, odnosno kao ulaz svakog regresora određenog zgloba jeste celokupna slika,
- nije neophodno eksplicitno dizajnirati reprezentaciju fičera i detektore za delove tela i nije neophodno eksplicitno dizajnirati topologiju modela i interakcije između zglobova, kao što je to bio slučaj kod metoda koje su se bazirale na grafičkom modelu, već se vrši obučavanje generičke CNN za ovaj problem.

Korišćenjem kaskade regresora postiže se uvećavanje preciznosti lociranog zgloba. Nakon inicijalne detekcije poze na osnovu cele slike, kaskadni DNN klasifikatori se obučavaju da poboljšaju predikcije zglobova koristeći delove veće rezolucije iz ulazne slike. [17]

Detekcija poze se obavlja iz nekoliko faza. Prva (inicijalna) faza prikazana je na slici 16 i predstavlja jedan DNN baziran regresor. Regresor se sastoji iz sedam slojeva u kome je plavom bojom označen konvolucioni sloj, a zelenom potpuno povezani sloj. Ulaz u prvi sloj je slika dimenzija 220x220 piksela puta 3 za svaki kanal boja, a izlaz iz poslednjeg sloja *2k* koordinata zglobova (*k* zglobova po dve koordinate). Svaki konvolucioni i potpuno povezani sloj se sastoji iz jedne linearne i jedne nelinearne (ReLU) transformacije. Na slici 16 su prikazani samo slojevi koji imaju parametre, a njihov ukupan broj je u ovom modelu 40M.



Slika 16: DNN baziran regresor, inicijalna faza [17]

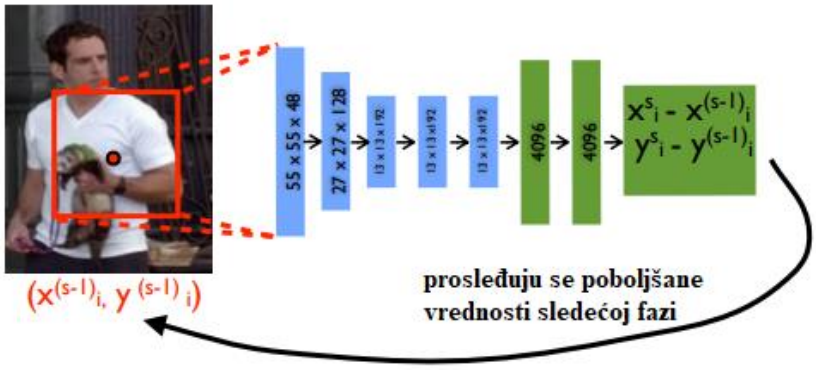
Potpuna arhitektura je prikazana sledećom formulom:

$$C(55 \times 55 \times 96) - LRN - P - C(27 \times 27 \times 256) - LRN - P - C(13 \times 13 \times 384) - C(13 \times 13 \times 384) - C(13 \times 13 \times 256) - P - F(4096) - F(4096),$$

gde *C* označava konvoluciju, *LRN* lokalni normalizacioni sloj, *P* puling sloj i *F* potpuno povezani sloj. [17]

Prednost inicijalne faze je što se procena zglobova izvršava na celoj slici i tako se oslanja na njen kontekst. Međutim ulazna slika je fiksnih dimezija 220x220 i samim tim donosi sa sobom previše detalja u mrežu. Pošto bi se povećanjem dimenzija slike povećao ionako veliki broj parametara, autori su kreirali i trenirali kaskadu regresora poze. Nakon inicijalne faze koja je odredila inicijalnu pozu tela slede podfaze koje se sastoje iz dodatnih DNN baziranih regresora koji su istrenirani da predviđaju pomeraj lokacija zglobova iz prethodne faze od njihovih stvarnih lokacija. Podfaze zapravo imaju cilj da "pročiste" i poboljšaju prethodno predviđenu pozu.

U podfazama se koriste prethodno predviđene lokacije zglobova kako bi se predviđanje fokusiralo na relevantne delove slike za taj zglob. Vrši se odsecanje glavne slike oko predviđene lokacije zgloba i prosleđuje regresoru pomeraja poze (eng. *pose displacement regressor*) na obradu. Na ovaj način regresor pomeraja ima uvid u veću rezoluciju relevantnog dela slike i može izvršiti preciznije predviđanje. [17] Primer jedne podfaze *s* prikazan je na slici 17.



Slika 17: DNN baziran regresor pomeraja poze [17]

Kaskada DNN baziranih regresora se sastoji iz fiksnog broja faza *S*. U svakoj fazi *s*, $s \in \{1,..., S\}$ se koristi ista mrežna arhitektura ali sa različito utreniranim mrežnim parametrom θs. Kako bi se obavilo poboljšanje zadate lokacije zgloba y_i razmatra se "granična kutija" (eng. *bounding box*) b_i oko zgloba, koja će iseći sliku oko y_i, predstavljena sledećom formulom:

$$\boldsymbol{b_i}\ \mathbf{(y; \sigma) = (}\boldsymbol{y_i}\mathbf{, \sigma*diam(y), \sigma*diam(y)),}$$

gde prvi parametar y_i označava centar, a sledeća dva parametra širinu i visinu (respektivno) "granične kutije". Širina i visina predstavljaju dijametar poze skaliran sa σ. Dijametar poze *diam(y)* je definisan kao distanca između suprotnih zglobova u odnosu na torzo, kao na primer levo rame i desni kuk, i zavisi od konkretne definicije poze i skupa podataka (*dataset-a*). [17]

Na samom kraju detektovana poza tela je predstavljena preko vektora poze (eng. *pose vector*) koji sadrži lokacije svih *k* detektovanih zglobova tela i predstavlja se formulom:

$\boldsymbol{y} = (\ldots, \boldsymbol{y_i^T}, \ldots)^{\boldsymbol{T}}$, $i \in \{1, \ldots, k\}$, gde y_i sadrži *x* i *y* koordinate.

Na slici 18 prikazan je rezultat prve tri faze kaskadnih regresora, gde se jasno uočava preciznije predviđena poza u svakoj narednoj fazi. Dakle može se zaključiti da je preciznost kod ovog pristupa proporcionalna broju faza *S*.



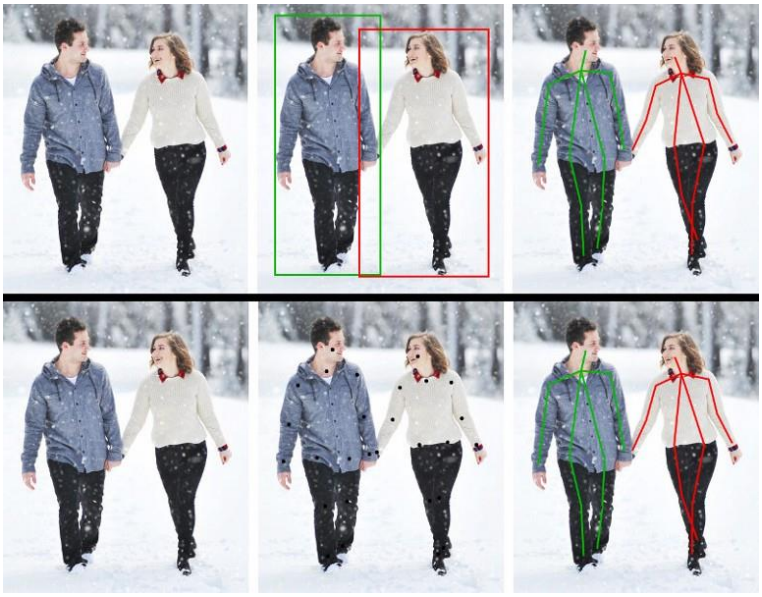
Slika 18: Predviđene poze (crveno) i stvarna poza (zeleno) u prve tri kaskadne faze kod DeepPose-a [17]

Tehnike dubokog učenja za detekciju poze tela

Kod detektovanja poze tela više ljudi na slici korišćenjem dubokog učenja mogu se izdvojiti dve tehnike:

- **top-down**
- **bottom-up**.

Kod top-down pristupa prvo se detektuju pojedinačne osobe, a nakon toga se vrši predikcija delova tela i određuje poza za svaku osobu posebno, slika 19 (gore). Kod bottom-up pristupa prvo se detektuju svi delovi tela svih osoba na osnovu ključnih tačaka, pa se nakon toga vrši združivanje i grupisanje delova koji pripadaju različitim osobama, slika 19 (dole). [2]



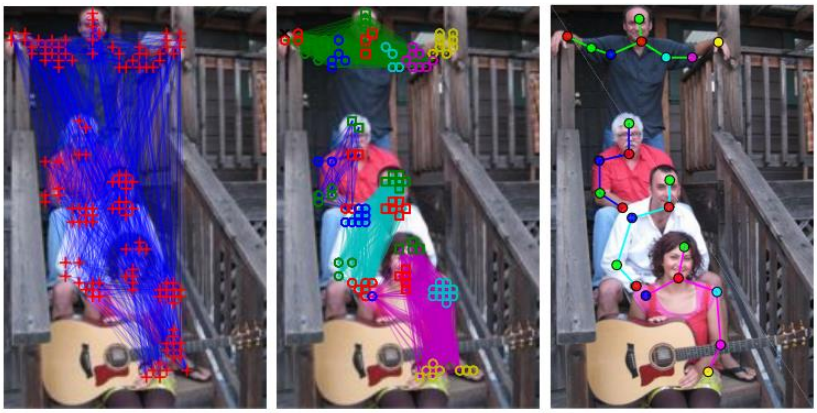
Slika 1919: Top-down (gore) i bottom-up (dole) metod 1 [2]

Top-down pristup je lakši za implementaciju jer je detekcija osoba na slici jednostavnija od algoritama za združivanje i grupisanje koji se koriste kod bottom-up pristupa, pa je zbog toga on korišćen u većini radova koji se bave ovom temom. Top-down pristup direktno zavisi od kvaliteta i preciznosti detektora osoba i ukoliko on ne detektuje osobe kako treba, naročito kada su osobe u neposrednoj blizini, doći će do grešaka koje se ne mogu izbeći. Još jedan nedostatak top-down pristupa je taj što je brzina obrade proporcionalna broju osoba na slici, jer se algoritam za detekciju poze mora primeniti za svaku osobu pojedinačno. Generalno se može reći da top-down pristup obezbeđuje veću preciznost, ali manju brzinu u poređenju sa bottom-up pristupom. Glavni razlog manje preciznosti kod bottom-up pristupa je ograničenje u rezoluciji, jer se kao ulaz u konvolucionu neuronsku mrežu koristi celokupna slika. U slučaju top-down pristupa slika se seče na delove oko detektovanih osoba koji se pojedinačno propuštaju kroz mrežu i na ovaj način je mreži dostupno mnogo više detalja i moguće je obaviti preciznije predviđanje.

Jedan od najpopularnijih modela za detekciju poza tela više ljudi na slici i koji koristi bottom-up pristup jeste ***OpenPose***, koji je ovde naveden kao primer, a u narednom poglavlju će biti detaljnije obrađen. U nastavku slede primeri **DeepCut** koji koristi bottom-up i **RMPE** koji koristi top-down pristup.

DeepCut je predstavnik bottom-up pristupa koji se koristi za detekciju poze više ljudi na slici. Ideja autora kod DeepCut-a bila je rešavanje tri definisana problema:

1. Pronaći skup *D* delova tela svih kandidata. Skup predstavlja moguće lokacije svih delova tela svih ljudi na slici.
2. Odrediti za svaku detektovanu lokaciju kojoj klasi *c* dela tela ona pripada. Klasa dela tela može biti: ruka, noga, grudi, rame...
3. Izdvajanje delova tela koji pripadaju jednoj osobi [12]



Slika 2020: Pregled metode, a) inicijalno detektovani i upareni delovi formiraju graf, b) delovi grafa koji pripadaju jednoj osobi, c) prikaz predviđenih skeleta [12]

Na početku, lociranje i grupisanje ljudskih delova se izvršava korišćenjem konvolucione neuronske mreže (CNN), a zatim se određivanje poze više ljudi rešava kao problem celobrojnog linearnog programiranja (eng. *Integer Linear Programming - ILP*).

Kako bi dobili dobar detektor delova, autori su prilagodili *Fast R-CNN* (FR-CNN) konvolucionu neuronsku mrežu. Mreža je nazvana AFR-CNN (eng. *Adapted Fast R-CNN*), a modifikovani delovi su generisanje predloga i veličina regiona otkrivanja. Generisanje predloga je od suštinske važnosti za rad FR-CNN, dok je otkrivanje delova tela izazovno zbog njihove male veličine i velike promenljivosti unutar klase. Za generisanje predloga, autori su koristili detektor baziran na deformabilnom modelu delova (eng. *Deformable Part Model – DPM*). Sakuplja se *K* najboljih detekcija od svih detektora, gde je jedan detektor zadužen za jedan deo, u zajednički skup od *N* predloga koji su nezavisni od dela i ti predlozi se koriste kao ulaz u AFR-CNN. Za slučaj jedne osobe na slici *N* je 2000, a kod više osoba 20 000. Korišćenje predloga za detekciju delova tela može biti neoptimalno, pa su autori razvili potpuno konvolucioni *VGGNet* za računanje ocena verovatnoće dela tela. [12]

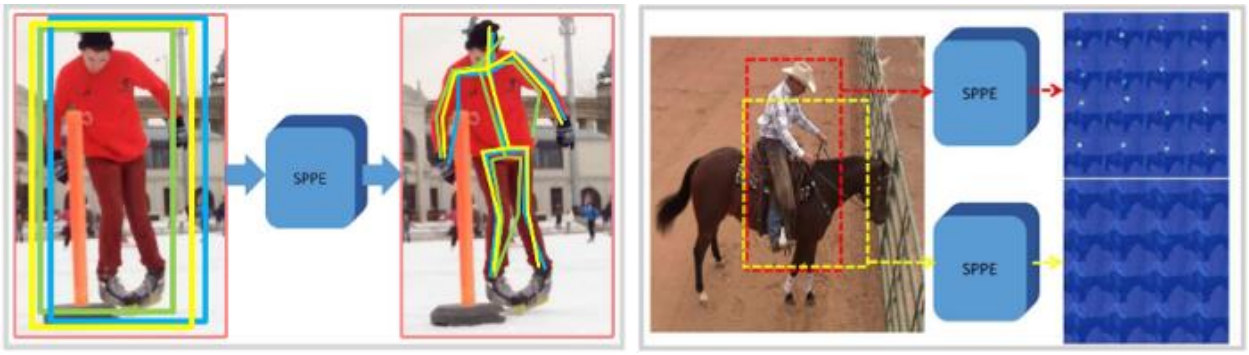
ILP se modelira razmatranjem trojki (x, y, z) binarnih slučajnih promenljivih u domenima:

$$x \in \{0,1\}^{D \times C}, y \in \{0,1\}^{\binom{D}{2}} \text{ and } z \in \{0,1\}^{\binom{D}{2} \times C^2}$$

Ukoliko posmatramo dva kandidata za deo tela *d* i *d'* iz skupa *D* kandidata za delove tela i dve klase *c* i *c'* iz skupa klasa *C*, gde su kandidati dobijeni iz konvolucione neuronske mreže, može se izdvojiti sledeći skup iskaza:

- ako je *x(d, c)=1*, tada kandidat *d* pripada klasi *c*,
- *y(d, d')=1*, tada kandidati *d* i *d'* pripadaju istoj osobi,
- ako je *z(d, d', c, c')=1*, gde je *z(d, d', c, c')= x(d, c)* x(d', c')* y(d, d')*, tada kandidat *d* pripada klasi *c*, kandidat *d'* pripada klasi *c'* i kandidati *d* i *d'* pripadaju istoj osobi. [2]

RMPE – *Regional Multi-person Pose Estimation* (**AphaPose**) je popularan model koji koristi **top-down** pristup pri detekciji poze tela. Autori ovog modela navode da top-down metod obično zavisi od preciznosti detektora osoba, pošto se detekcija poze vrši u regionu u kome se osoba nalazi, pa zbog toga greške u lokalizaciji i dupliranje graničnog okvira u kome se nalazi jedna osoba, može izazvati rad algoritma za detektovanje poze neoptimalnim. [2]



Slika 21: Efekti grešaka pri dupliranju predikcije (levo) i lokalizaciji osobe (desno) [2] [RMPE]

Kako bi se prevazišli ovi problemi, autori su predložili korišćenje simetrične mreže prostornih transformatora (eng. *Symmetric Spatial Transformer Network* - ***SSTN***) za ekstrakciju regiona visokog kvaliteta jedne osobe iz netačnog graničnog okvira. **SPPE** (*Single Person Pose Estimator*) se koristi da proceni skelet ljudske poze osobe u prethodno izdvojenom regionu. Zatim se koristi **SDTN** (eng. *Spatial De-Transformer Network*) da mapira procenjenu ljudsku pozu nazad u originalni koordinatni sistem slike. Konačno, tehnika parametarskog suzbijanja ne-maksimalnog potiskivanja (eng. *Non-Maximum Suppression* - ***NMS***) se koristi za rešavanje problema odbacivanja redundantnih poza. [2]

Primena detekcije poze tela

Procena poze tela je široko korišćena u brojnim aplikacijama iz različitih oblasti. Jedna od primena jeste korišćenje detekcije poze kako bi se detektovale aktivnosti, gestikulacije ili hod ljudi. To se primenjuje u aplikacijama koje se bave sigurnošću i nadzorom, kao na primer za detekciju da li je osoba pala ili je bolesna, aplikacijama koje razumeju govor celokupnog tela (npr. znakove saobraćajnog policajca), itd. [2] Zanimljiva primena jeste u filmskoj industriji gde se vrši detekcija poze glumca uz pomoć određenih markera na njegovom telu kako bi se pokret preneo na animirane likove u filmu (slika 22).



Slika 21: Primena detekcije poze i facijalnih ekspresija u animiranim filmovima [10]

Još jedna primena gde se detekcija poze može koristiti je u robotici. Umesto ručnog programiranja robota da prate određene putanje i vrše određene pokrete, roboti mogu biti napravljeni da prate putanje skeleta ljudske poze pri izvođenju određene akcije. Čovek instruktor može efikasnije učiti robota određenim akcijma jednostavno demonstrirajući mu njeno izvođenje. Robot će zatim izvršiti izračunavanje kako da pokrene svoje aktuatore kako bi izvršio istu akciju. [2]

U industriji video igara, praćenje pokreta je takođe pronašlo svoju primenu. Konzole koriste dubinske kamere kako bi detektovale pokrete igrača i koristile ih u interaktivnim igrama. Na primer Kinect vrši detekciju 3D poza igrača i koristi ih da prikaže akcije virtuelnih karaktera. [2]



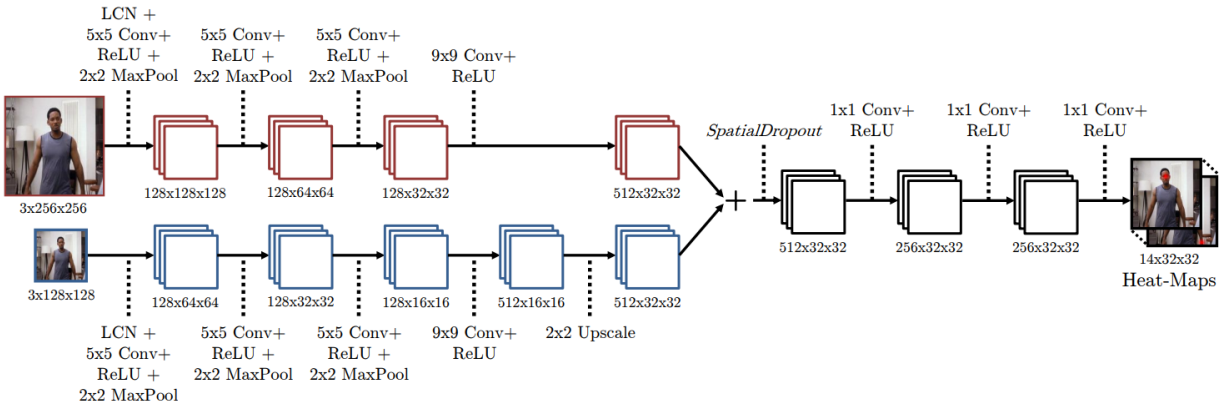
Slika 23: Primer Kinect interaktivne igre [11]

Proširena realnost je takođe jedno od polja gde se detekcija poze može primeniti. Detekcija poze otvara mogućnost kreiranja realističnijeg i interaktivnijeg doživljaja proširene realnosti. Ono što proširena realnost pruža jeste prikazivanje digitalnih (virtuelnih) objekata u stvarnom okruženju. Ukoliko se iskoristi lokalizacija i praćenje stvarnih objekata u realnom prostoru, tada se njihovi pokreti i ponašanje mogu preneti na virtuelne objekte učinivši ih na taj način realističnijim. [4]

Postojeći modeli za detekciju poze tela

Efikasna lokalizacija objekata pomoću konvolucionih neuronskih mreža

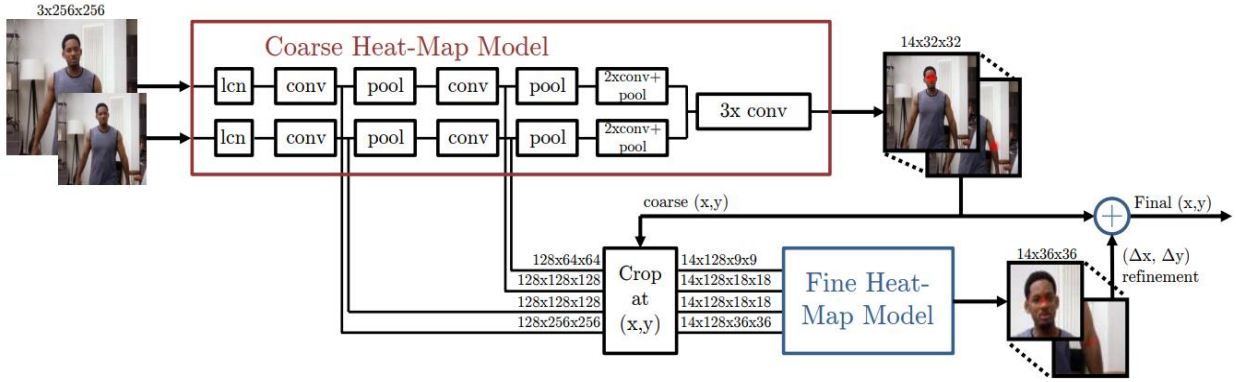
Sledeći rad pod nazivom Efikasna lokalizacija objekata pomoću konvolucionih mreža [18] prezentovan na CVPR konferenciji 2015. godine predstavio je ConvNet arhitekturu za efikasnu lokalizaciju ljudskih zglobova na RGB slikama koja postiže visoku prostornu tačnost bez značajnih računskih troškova. Ovaj model je omogućio korišćenje veće količine puling slojeva za efikasnija izračunavanja, uz zadržavanje visoke prostorne preciznosti. Koristi se kaskada grubih i finih ConvNet modela. Autori su predložili novu arhitekturu zajedničkih fičera koja omogućava treniranje oba modela za detekciju zglobova radi poboljšanja performansi generalizacije i koja uzorkuje podskup ulaznih fičera za poboljšanje performansi u toku izvršavanja. Koristi se ConvNet arhitektura sa više rezolucija (slika 24) da bi se implementirao detektor na bazi klizajućeg prozora sa kontekstima koji se preklapaju da bi se proizvele grube izlazne toplotne mape. [18]



Slika 2422: Višerezolucioni detektor na bazi klizajućeg prozora sa preklapanjem konteksta, korišćen nad FLIC skupom podataka [18]

Model grubih regresionih toplotnih mapa kao ulaz ima RGB Gausovu piramidu sa 3 nivoa i kao izlaz daje toplotnu mapu za svaki zglob (na slici su prikazana samo 2 nivoa). Toplotna mapa opisuje verovatnoću svakog piksela da se na njemu nalazi prostorna lokacija zgloba. Koriste se ulazne rezolucije od 320x240 i 256x256 piksela za FLIC i MPII skupove podataka, redom. Prvi sloj mreže je sloj lokalne kontrastne normalizacije (eng. *local-contrast-normalization* - LCN) sa istim filtrom u svakoj od tri rezolucione banke. Svaka LCN slika se zatim unosi u sedmostepenu konvolucionu mrežu sa više rezolucija (11 stepena za model MPII skupa podataka). Zbog prisustva pulinga, izlaz toplotne mape je na manje rezolucije od ulazne slike. Poslednje 4 faze (ili 3 faze za model MPII skupa podataka) efikasno simuliraju potpuno povezanu mrežu. Model je poboljšan dodavanjem dodatnog napuštajućeg sloja (eng. *dropout layer*) pre prvog 1x1 konvolucionog sloja na slici 18 označen kao *SpatialDropout*. Uloga napuštanja (*dropout-a*) je poboljšanje performanse generalizacije sprečavajući da aktivacije postanu snažno povezane, što dovodi do pretreniranosti (eng. *over-training*). [18]

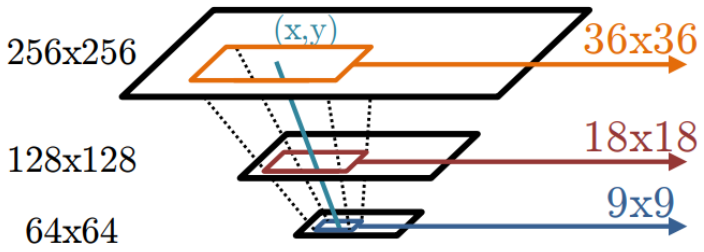
Glavni cilj ovog rada je oporavak prostorne tačnosti izgubljene usled pulinga upotrebom dodatnog ConvNet-a za pročišćavanje rezultata lokalizacije grube toplotne mape. Međutim, za razliku od standardnog kaskadnog modela ovde se ponovo koriste postojeći konvolucioni fičeri.



Slika 25: Pregled celokupne kaskadne arhitekture [18]

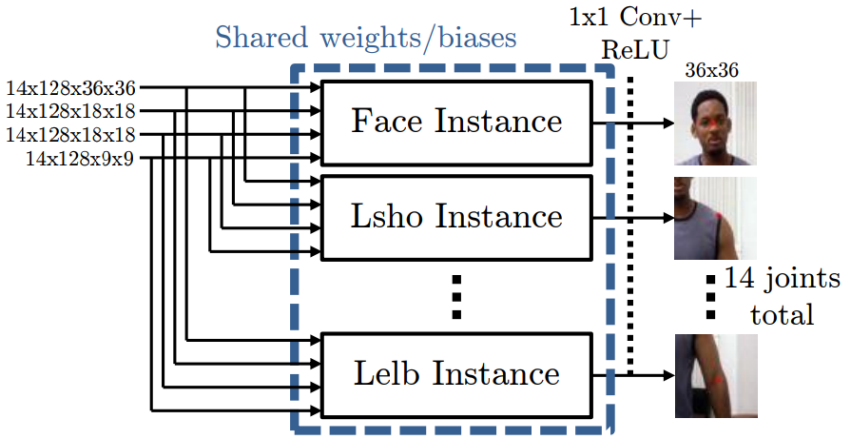
Kompletna arhitektura sistema prikazana je na slici 25. Sastoji se iz delova modela zasnovanih na toplotnoj mapi za grubu lokalizaciju, modula za uzorkovanje i isecanje konvolucionih fičera na određenim lokacijama (x, y) za svaki zglob, kao i dodatnog konvolucionog modela za finu toplotnu mapu. Predikcija zglobova iz ulazne slike se obavlja na sledeći način: slika se propušta kroz model grube toplotne mape gde se vrši predikcija svih lokacija zglobova (x, y) iz maksimalnih vrednosti na toplotnoj mapi svakog zgloba. Zatim se koristi gruba (x, y) lokacija za uzorkovanje i isecanje fičera sa slika nakon prva 2 konvoluciona sloja (za sve rezolucione banke) za svaku od lokacija zglobova. Zatim se propuštaju ovi fičeri kroz model fine toplotne mape da bi se proizveli pomeraji (Δx, Δy) unutar isečenog podprozora. Na kraju se pomeraji dodaju na grube lokacije kako bi se proizvele preciznije konačne (x, y) lokalizacije za svaki zglob. [18]

Slika 26 prikazuje funkcionalnost modula za odsecanje za jedan zglob. Ovaj modul jednostavno odseca prozor centriran na grubu lokaciju zgloba (x, y) na svakoj rezolucionoj mapi fičera, pritom održavajući kontekstnu veličinu prozora konstantnim, skaliranjem isečene površine na svakom višem nivou rezolucije. [18]



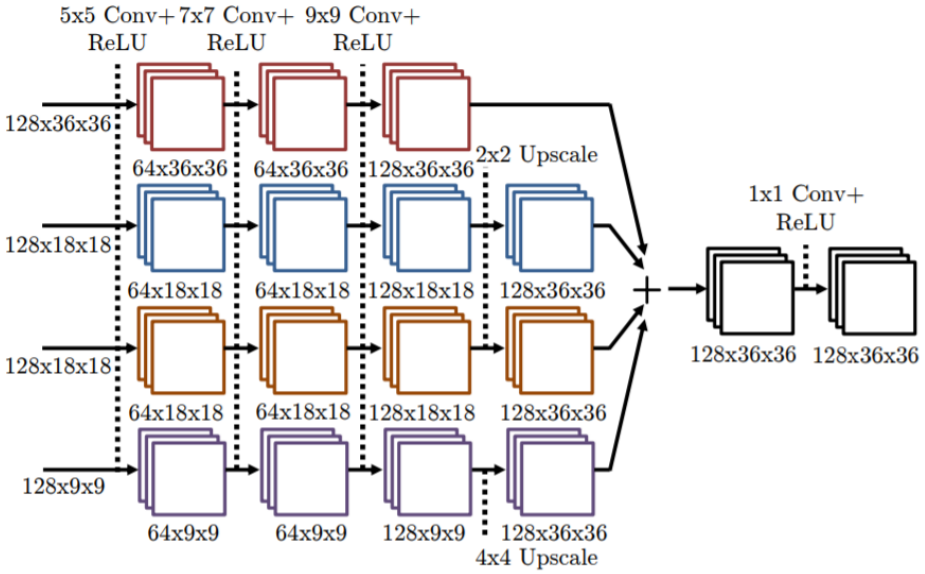
Slika 26: Funkcionisanje modula za odsecanje za jedan zglob [18]

Na slici 27 je prikazana arhitektura modula fine toplotne mape. Ovaj modul se sastoji iz 7 podmreža (14 podmreža za MPII skup podataka) gde je svaka mreža zadužena za jedan zglob i mogu raditi nezavisno. Sve mreže koriste iste parametre kako bi se smanjio ukupan broj parametara i izbegla pretreniranost. Na izlazu iz svake mreže primenjuje se 1x1 konvolucija, ali sada sa različitim parametrima kako bi na izlazu bile toplotne mape za svaku tačku bolje rezolucije. Cilj ovog konvolucionog sloja je da kreira finalne detekcije svih zglobova. [18]



Slika 27: Model fine toplotne mape za 14 zglobova [18]

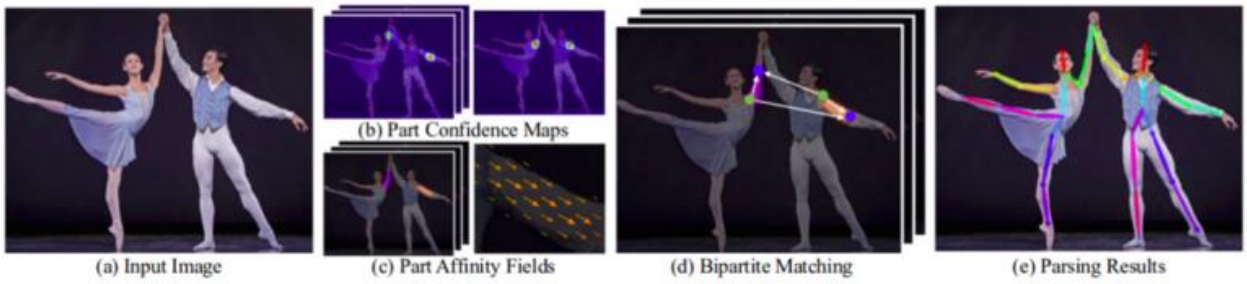
Svaka instanca podmreže na slici 22 je ConvNet od 4 sloja, kao što je prikazano na slici 28. Budući da su ulazne slike različitih rezolucija i potiču iz različitih dubina u modelu grube toplotne mape, ulazni fičeri se tretiraju kao odvojene rezolucione banke i primenjuje se konvolucija iste veličine na svaku banku. Nadograđuju se fičeri niže rezolucije da bi se doveli u kanoničku rezoluciju i primenjuju aktivacije na fičer mape, a zatim se primenjuje, gore navedena, 1x1 konvolucija nad izlaznim fičerima. [18]



Slika 28: Mreža fine toplotne mape za jedan zglob [18]

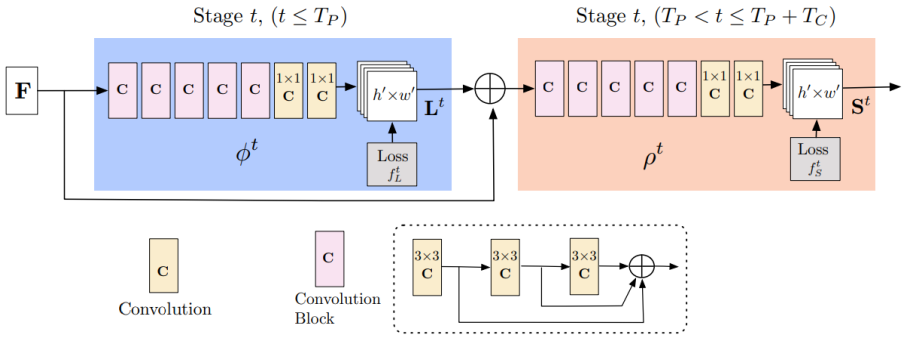
OpenPose

OpenPose je najpopularniji i najkorišćeniji model za detekciju 2D ljudske poze više ljudi na slici u realnom vremenu, objavljen 2019. godine. Kao i većina bottom-up pristupa, OpenPose prvo detektuje ključne tačke (*keypoints*) svih ljudi na slici pa nakon toga određuje kojoj osobi prikada koja tačka. Autori [3] predstavljaju metod u kome koriste neparametarski prikaz, koga nazivaju poljima afiniteta dela (eng. *Part Affinity Fields* – ***PAFs***, *slika 29c*), da nauči da povezuje delove tela sa konkretnim osobama na slici. PAFs predstavlja 2D vektor polja koji određuju lokaciju i orijentaciju udova u domenu slike. Razlika kod OpenPose i prethodnog rada [19] koji takođe koristi PAFs u detekciji poze, jeste što se ne vrši poboljšanje ("prečišćavanje") i PAFs i lokacija delova tela, već samo PAFs što rezultuje povećanjem i preciznosti i performansi u realnom vremenu. Samim tim što se ne vrši poboljšanje detektovanih lokacija delovi CNN koji su ranije bili zaduženi za tu obradu su izbačeni, dok je samoj mreži povećana dubina. Ovako izmenjena mreža je povećala brzinu obrade za 200%, a preciznost za 7%. Još jedna novina koju ovaj model predstavlja jeste korišćenje prvog kombinovanog detektora ključnih tačaka tela i stopala, koji unosi dodatnu preciznost u proceni poze.



Slika 23: : Koraci pri obradi slike kod OpenPose [3]

Ulaz u OpenPose sistem je RGB slika dimenzija *w* x *h* (slika 29a), a izlaz 2D lokacije ključnih tačaka svih osoba na slici (slika 29e). Na početku slika se analizira koristeći prvih 10 slojeva VGG-19 mreže (u nekim varijacijama i ResNet) i generišu se fičer mape ***F*** koje se prosleđuju prvoj fazi na obradu. Celokupna obrada se sastoji iz nekoliko faza ***Tp*** u kojima se iterativno određuju polja afiniteta koja opisuju asocijaciju između dva dela tela (slika 30 plava boja) i nekoliko faza ***Tc*** u kojima se vrši detekcija mapa poverenja, eng. *confidence maps* (primer na slici 29b i arhitektura za iterativno određivanje na slici 30 bež bojom). Ovakav iterativni pristup poboljšava predviđanje u sukcesivnim fazama uz posrednički nadzor u svakoj fazi. [3]



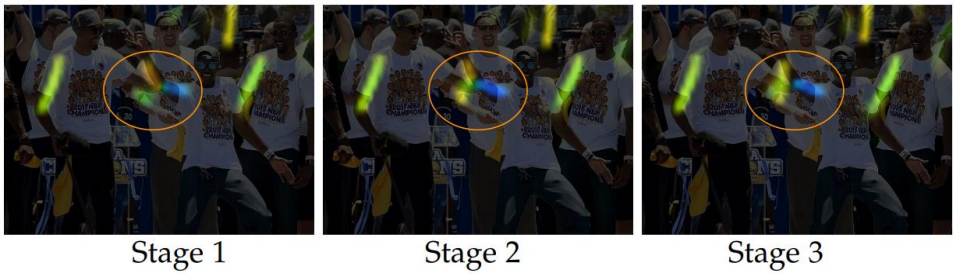
Slika 24: Arhitektura višefazne CNN [3]

U arhitekturi sa slike 30 može se videti korišćenje konvolucionih blokova koji se sastoje iz primene 3 uzastopna 3x3 kernela čiji se izlazi na kraju bloka konkateniraju. Jedan blok zapravo zamenjuje primenu jedne 7x7 konvolucije iz prethodnog rada istih autora [19]. Ovakav princip gde se konvolucija većim filtrom zamenjuje uzastopnom primenom 3x3 filtra, primenjen prvi put kod *VGGNet-a,* umanjuje broj operacija, pa samim tim i ubrzava izračunavanje. Takođe u odnosu na prethodnu arhitekturu, broj nelinearnih slojeva je utrostručen i time mreža može zadržati fičere i nižeg i višeg nivoa.

Mreža u prvoj fazi proizvodi skup polja afiniteta dela (PAFs) $\boldsymbol{L^1} = \varphi^1(\boldsymbol{F})$, gde φ^1 predstavlja zaključivanje u fazi 1. U svakoj sledećoj fazi, predviđanja iz prethodne faze i originalni fičeri slike $\boldsymbol{F}$ se konkateniraju i koriste kako bi se kreirale poboljšane predikcije. Skup PAFs u fazi t:

$$\boldsymbol{L}^t = \varphi^t(\boldsymbol{F}, \boldsymbol{L}^{t-1}), \ \forall 2 \leq t \leq \boldsymbol{Tp},$$

gde se φ^t odnosi na CNN za zaključivanje u fazi t i $\boldsymbol{Tp}$ ukupan broj PAF faza. [3] Na slici 31 prikazan je postupak poboljšanja polja afiniteta desne nadlaktice kroz prve tri faze. Mogu se uočiti grešake u određivanju zbog preklapanja delova tela u prvoj fazi, a koje se u narednim fazama uspešno ispravljaju. Različitim bojama su predstavljene različite orijentacije vektora afiniteta polja.



Slika 25: Polja afiniteta desne nadlaktice kroz 3 faze [3]

Nakon $\boldsymbol{Tp}$ faza u kojima se dobija PAFs, kreće narednih $\boldsymbol{Tc}$ faza u kojima se iterativno detektuju mape poverenja počevši od najbolje PAF predikcije:

$$\boldsymbol{S}^{Tp} = \rho^t(\boldsymbol{F}, \boldsymbol{L}^{Tp}), \ \forall t = \boldsymbol{Tp},$$

a nadalje:

$$\boldsymbol{S}^t = \rho^t(\boldsymbol{F}, \boldsymbol{L}^{Tp}, \boldsymbol{S}^{t-1}), \ \forall \ Tp < t \leq Tp + Tc,$$

gde se ρ^t odnosi na CNN za zaključivanje u fazi t. [3]

Ovakvim tokom obrade postignuto je da "prečišćene" predikcije polja afiniteta poboljšavaju rezultate mapa poverenja. Izlaz iz ove mreže (nakon $\boldsymbol{Tp}$ + $\boldsymbol{Tc}$ faza) se sastoji iz skupa 2D mapa poverenja lokacija delova tela $\boldsymbol{S}$ (slika 29b) i skupa 2D vektora polja afiniteta delova tela (PAFs) $\boldsymbol{L}$ (slika 29c), koji predstavljaju stepen asocijacije između delova. Skup $\boldsymbol{S}$ se sastoji iz ukupno $\boldsymbol{J}$ mapa poverenja, jedne za svaki deo tela:

$$\boldsymbol{S} = (\boldsymbol{S_1}, \boldsymbol{S_2}, \ldots, \boldsymbol{S_J}), \text{ gde } \boldsymbol{S}_j \in \mathbb{R}^{w \times h}, \ j \in \{1 \ldots J\}.$$

Skup $\boldsymbol{L}$ se sastoji iz ukupno $\boldsymbol{C}$ vektora polja, jedan za svaki ud:

$$\boldsymbol{L} = (\boldsymbol{L_1}, \boldsymbol{L_2}, \ldots, \boldsymbol{L_C}), \text{ gde } \boldsymbol{L}_c \in \mathbb{R}^{w \times h \times 2}, \ c \in \{1 \ldots C\}.$$

Mape poverenja $\boldsymbol{S}$ i PAFs $\boldsymbol{L}$ se prosleđuju i obrađuju u poslednjem koraku koristeći *greedy bipartite matching algorithm* (slika 29d) kako bi se dobile 2D ključne tačke svih ljudi na slici. [3]

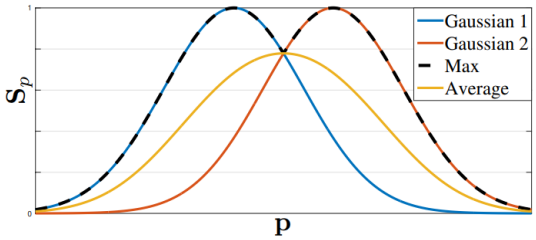
Svaka mapa poverenja predstavlja 2D reprezentaciju verovatnoće da se na zadatom pikselu nalazi određeni deo tela. Na primer u idealnom slučaju, ukoliko se na slici nalazi samo jedan čovek očekuje se samo jedan pik (maksimum verovatnoća) za svaku mapu poverenja dela tela koji je vidljiv, dok u slučaju više osoba može postojati više pikova, odnosno jedan pik po vidljivom delu tela j za svaku osobu k. Mape poverenja za svaku osobu k i svaki deo tela j definisane su kao:

$$\mathbf{S}^*_{j,k}(\mathbf{p}) = \exp\left(-\frac{||\mathbf{p} - \mathbf{x}_{j,k}||_2^2}{\sigma^2}\right),$$

to je Gausova kriva sa postepenim promenama gde σ kontroliše širenje pika (vrha). Predviđeni pik iz mreže je agregacija individualnih mapa poverenja max operatorom,

$$\mathbf{S}^*_j(\mathbf{p}) = \max_k \mathbf{S}^*_{j,k}(\mathbf{p}).$$

Koristi se maksimum mapa poverenja umesto proseka tako da preciznost približnih pikova ostaje različita, što je ilustrovano na slici 32. [3]

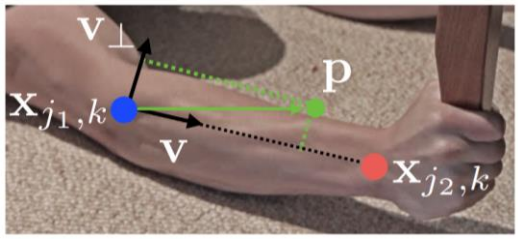


Slika 26: Korišćenje maksimalnih vrednosti pikova umesto prosečne vrednosti [3]

Polje afiniteta dela tela je potrebno posebno kod detekcije poza više osoba na slici gde je potrebno korektno odrediti kom telu pripada određeni deo tela, jer postoji više glava, ramena, ruka.. Problem nastaje kada su svi delovi usko grupisani. Ove probleme rešavaju polja afiniteta tako što sadrže i lokacione i orijentacione informacije preko regija koje označavaju čovekov ekstremitet. Za svaki ekstemitet postoji PAF koji predstavlja 2D vektor polja. Za svaki piksel u regiji u kojoj se nalazi neki ekstremitet, 2D vektor sadrži smer (direkciju) te tačke od jedne do druge strane ekstremiteta. Svaki tip ekstremiteta ima odgovarajući PAF koji spaja njegova dva zgloba. Na slici 33 prikazan je primer podlaktice na kojoj $x_{j1,k}$ i $x_{j2,k}$ predstavljaju pozicije zglobova j_1 i j_2 ekstremiteta c osobe k na slici. Ukoliko tačka $\boldsymbol{p}$ pripada regiji na kojoj se nalazi ekstremitet, vrednost $L^*_{c,k}(\boldsymbol{p})$ predstavlja jedinični vektor koji je usmeren od j_1 prema j_2. Za sve ostale tačke koje se ne nalaze na regiji jedinični vektor je 0.

$$\mathbf{L}^*_{c,k}(\mathbf{p}) = \begin{cases} \mathbf{v}, & \text{ako je } \boldsymbol{p} \text{ na ekstremitetu } c,k \\ \mathbf{0}, & \text{u ostalim slučajevima} \end{cases}$$

gde je $\mathbf{v} = (x_{j2,k} - x_{j1,k}) / \|x_{j2,k} - x_{j1,k}\|_2$ jedinični vektor u pravcu ekstremiteta. [3]



Slika 27: Određivanje polja afiniteta na primeru podlaktice [3]

Mape poverenja sadrže veliki broj zglobova, naročito ako se na slici nalazi više ljudi. Veliki broj zglobova otvara mogućnost postojanja velikog broja udova (ekstremiteta), pa je potrebno odrediti koji su udovi stvarni, a koji ne. Prvo se pribavlja skup kandidata ljudskih zglobova (slika 34a):

$$\boldsymbol{D_j} = \{\, \boldsymbol{d_j^m} : \text{za } j \in \{1 \ldots J\}, m \in \{1 \ldots N_j\}\},$$

gde je N_j broj kandidata za zglob j i $\boldsymbol{d_j^m} \in \mathbb{R}^2$ lokacija m-tog detektovanog kandidata za zglob j. Zatim se definiše promenljiva $z_{j1\,j2}^{mn} \in \{0, 1\}$ koja određuje da li su dva detektovana kandidata $\boldsymbol{d_{j1}^m}$ i $\boldsymbol{d_{j2}^n}$ povezana, a cilj je zatim pronaći optimalan način da se izračuna vrednost promenljive za sve moguće veze:

$$\boldsymbol{Z} = \{\, \boldsymbol{z_{j1\,j2}^{mn}} : \text{za } j_1, j_2 \in \{1 \ldots J\}, m \in \{1 \ldots N_{j1}\}, n \in \{1 \ldots N_{j2}\}\}.$$

Ukoliko se posmatra potpuno povezani graf (slika 34b) čvorovi su detektovani zglobovi kandidati D_{j1} i D_{j2}, a potezi su sve moguće veze između detektovanih zglobova, odnosno mogući delovi tela (udovi). Svaki poteg ima svoju težinu na osnovu jednačine:

$$E = \int_{u=0}^{u=1} \mathbf{L}_c\left(\mathbf{p}(u)\right) \cdot \frac{\mathbf{d}_{j_2} - \mathbf{d}_{j_1}}{||\mathbf{d}_{j_2} - \mathbf{d}_{j_1}||_2} du, \quad (1)$$

gde $\mathbf{p}(u)$ interpolira pozicije dva dela tela $\boldsymbol{d_{j1}}$ i $\boldsymbol{d_{j2}}$,

$$\mathbf{p}(u) = (1 - u)\, \boldsymbol{d_{j1}} + u\boldsymbol{d_{j2}}.$$

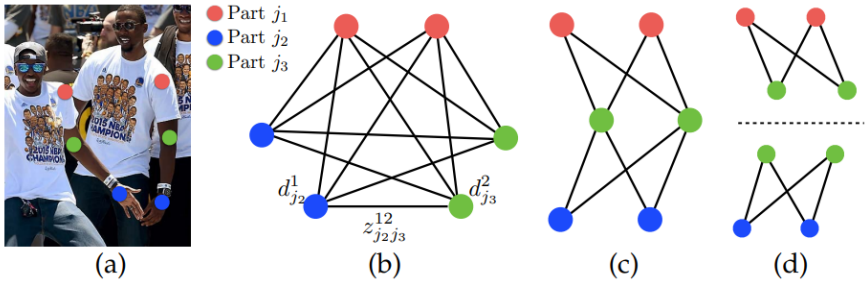
Podudaranje u dvodelnom grafu (eng. *bipartite graph matching*) je podskup potega izabranih na taj način da ni jedna dva potega ne dele isti čvor. Cilj je pronaći podudaranje sa maksimalnom težinom za izabrani poteg preko jednačina:

$$\max_{\mathcal{Z}_c} E_c = \max_{\mathcal{Z}_c} \sum_{m \in \mathcal{D}_{j_1}} \sum_{n \in \mathcal{D}_{j_2}} E_{mn} \cdot z_{j_1 j_2}^{mn}, \quad (2)$$

$$\forall m \in \mathcal{D}_{j_1}, \sum_{n \in \mathcal{D}_{j_2}} z_{j_1 j_2}^{mn} \leq 1, \quad (3)$$

$$\forall n \in \mathcal{D}_{j_2}, \sum_{m \in \mathcal{D}_{j_1}} z_{j_1 j_2}^{mn} \leq 1, \quad (4)$$

gde je E_c ukupna težina podudaranja sa udom tipa c, Z_c je podskup Z za tip uda c i E_{mn} je afinitet dela tela između zglobova d_{j1}^m i d_{j2}^m dobijen iz jednačine (1). Jednačine (3) i (4) obezbeđuju da ni jedna dva potega ne dele isti čvor, odnosno da na primer dve nadlaktice ne dele isti čvor koji predstavlja rame. Kod detekcije poza više ljudi na slici, određivanje Z predstavlja K-dimenzioni problem uparivanja koji je NP-težak. [3]



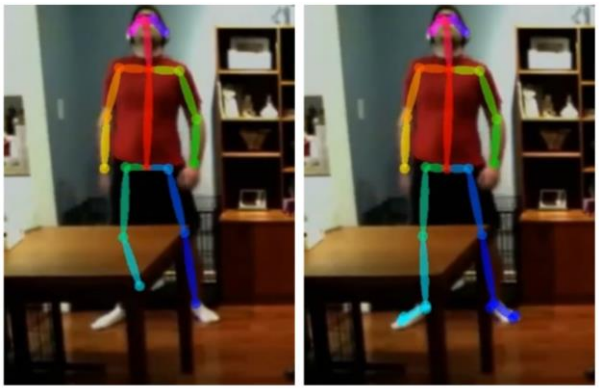
Slika 28: Podudaranja u grafu, (a) originalna slika sa detektovanim zglobovima, (b) potpuno povezan graf, (c) struktura stabla, (d) skup dvodelnih grafova [3]

Kako je izračunavanje nad potpuno povezanim grafom neoptimalno i previše zahtevno, uvode se dve optimizacije. Prva je da se umesto kompletnog grafa izabere minimalan broj potega koji čine rastegnuto (sveobuhvatno) stablo skeleta ljudske poze (slika 34c), a druga je da se problem podudaranja nad prethodno dobijenim stablom podeli u skup podproblema podudaranja u dvodelnim graforima i onda pronađe podudaranje u susednim čvorovima stabla (slika 34d). Razlog ovih optimizacija leži u tome što su veze između dva susedna čvora stabla eksplicitno modelovane preko polja afiniteta delova tela (PAFs). Nakon ovih transformacija, optimizacija je jednostavno razložena kao:

$$\max_{\mathcal{Z}} E = \sum_{c=1}^{C} \max_{\mathcal{Z}_c} E_c$$

Kandidati za vezu ekstremiteta se zatim dobijaju za svaki tip ekstremiteta nezavisno koristeći gore navedene jednačine (2-4). Sa svim kandidatima veze ekstemiteta, mogu se sakupiti veze koje dele iste zglobove u poze celih tela svih ljudi. [3]

Ovaj model takođe koristi i suvišne (redundantne) PAF veze, kao što su veze između ušiju i ramena, ili ručnih zglobova i ramena, kako bi povećale preciznost u detekciji poza na slikama gde su osobe veoma blizu ili se preklapaju. Još jedna stvar koju autori uvode u ovom radu jeste korišćenje kombinovanog modela, gde se pored detekcije tela dodatno koristi i detekcija stopala. Ovako kombinovana detekcija takođe utiče na povećanje preciznosti, naročito kod detekcije skočnih zglobova kao što je to prikazano na slici 35. Na levoj slici koristi se samo detekcija tela i vidi se greška pri detekciji desnog skočnog zgloba, dok je na desnoj slici prikazano korišćenje kombinovanog modela gde se vidi da je sada skočni zglob, na osnovu detektovanog desnog stopala, precizno detektovan.



Slika 29: Deteckija poze bez korišćenja (levo) i sa korišćenjem (desno) kombinovanog modela [3]

OpenPose je treniran i testiran nad tri skupa podataka (eng. *datasets*) MPII, COCO i COCO + stopala. Skup podataka za stopala koji se koristi, autori su sami kreirali izdvojivši skup od 15K primera ključnih tačaka iz COCO dataset-a. Na slici 36 mogu se videti primeri ključnih tačaka koji se dobijaju kao izlaz mreža treniranih nad različitim skupovima podataka.



(a) MPII (b) COCO (c) COCO+Foot

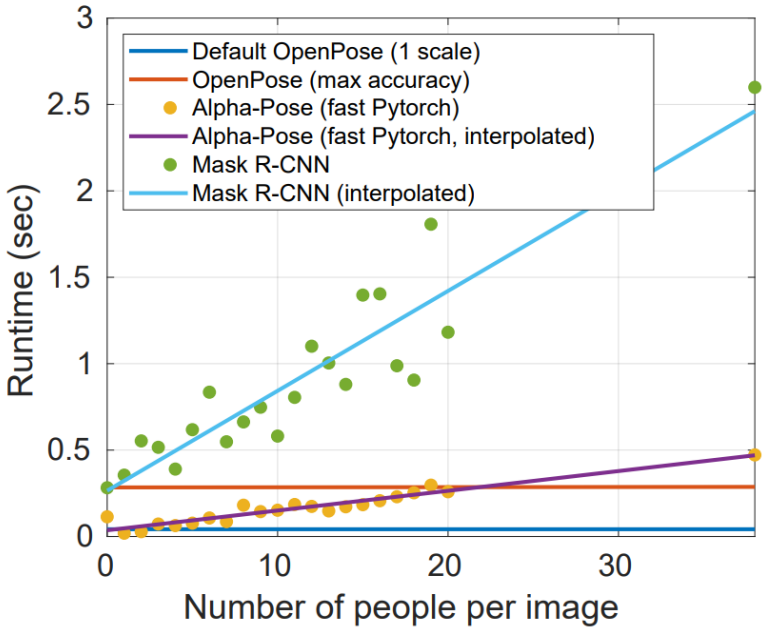
Slika 30: Prikaz ključnih tačaka zasnovanih na 3 različita skiupa podataka [3]

Arhitekture dubokih neuronskih mreža donose sa sobom veću preciznost u detekciji ali i veliki broj potrebnih operacija, kao i veće zauzeće memorije. Međutim broj operacija se može redukovati korišćenjem grafičkog procesora (GPU-a) za njihovu paralelizaciju. OpenPose koristi ovakav način ubrzanja tako što koristi CUDA i OpenCL kompatibilne grafičke procesore u ove svrhe. Ovakvim ubrzanjem može se postići oko 22 FPS (frejma po sekundi) na mašinama koje imaju Nvidia GTX 1080Ti, dok je sama preciznost idalje visoka. Autori su izvršili testiranje i upoređivanje modela treniranih nad različitim skupovima podataka njihovim izvršavanjem na sistemu sa CUDA i na sistemu koji koristi samo CPU za obradu. Rezultati su prikazani u tabeli 1 i može se zapaziti da model koji koristi telo plus noge pored toga što ima veću preciznost ima i oko 2 puta brže izvršenje od ostala dva originalna modela uz korišćenje grafičkog procesora, dok je u verziji gde se koristi samo CPU brzina ovog modela zapravo 5 puta sporija.

Model	**CUDA + CPU**	**CPU**
Originalni MPII model	73 ms	2309 ms
Originalni COCO model	74 ms	2407 ms
Model telo + stopala	36 ms	10396 ms

Tabela 1: Razlika izvršenja između tri modela OpenPose-a sa CUDA i samo CPU verzijom, na sistemu sa NVIDIA GeForce GTX-1080Ti grafičkom karticom i intel i7-6850K procesorom [3]

Autori takođe predstavljaju i poređenje OpenPose-a sa Mask R-CNN i AlphaPose koji su predstavnici top-down pristupa u detekciji poze. Analiza je izvršena korišćenjem istih slika za svaki algoritam i ponovljena 1000 puta na sistemu sa Nvidia 1080Ti i CUDA8, a prosečna vrednost je prikazana na grafikonu 1. Sa grafikona se može videti da je proces zaključivanja (određivanja) poza invarijantan na promenu broja osoba na slici kod OpenPose-a i da je vreme konstantno. U slučaju Mask R-CNN i AlphaPose brzina obrade raste sa porastom broja ljudi na slici. [3]



Grafikon 1: Poređenje zaključivanja između OpenPose, Mask R-CNN i AlphaPose [3]

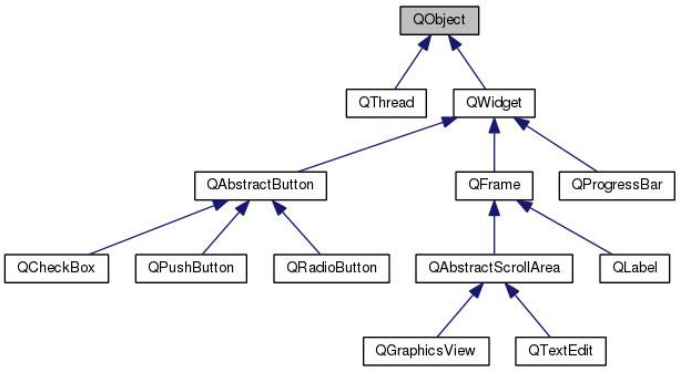
OpenPose je objavljen kao biblioteka otvorenog koda (eng. *Open Source library*) koja se može koristiti na različitim platformama, *Ubuntu, Windows, MacOSX* i ugrađeni sistemi (npr. Nvidia Tegra TX2). Biblioteka podržava različite hardvere kao što su CUDA GPUs, OpenCL GPUs i sistemi samo sa CPU. Moguće su detekcije poze tela, stopala, šaka i lica, a korisniku se pruža mogućnost da izabere šta mu je potebno. Takođe treba napomenuti da je biblioteka široko rasprostranjena i korišćena u mnogim radovima i aplikacijama koje se bave ovom oblašću.

Aplikacija za asistenciju u vežbanju

Detekcija poze tela na osnovu slika je, kao što je prethodno opisano, problem koji je uspešno rešen korišćenjem dubokog učenja, odnosno korišćenjem dubokih konvolucionih neuronskih mreža. Korišćenje ovih mreža zahteva izvršenje velikog broja akcija, a sam taj proces detekcije može trajati i nekoliko sekundi. Kao što je pomenuto ranije, korišćenje grafičkog procesora i određene optimizacije mogu pomoći u redukovanju tog vremena. Kod korišćenja detekcije poze na osnovu slika ili prethodno snimljenih klipova često vreme obrade nije toliko presudno. Međutim, u aplikacijama koje rade sa video ulazom i detektuju pozu u realnom vremenu, obrada jedne slike (frejma) je od presudnog značaja. Obrada u realnom vremenu predstavlja veliki izazov i bila je predmet brojnih istraživanja poslednjih godina. Pojavili su se radovi koji modifikacijama mreže, boljim korišćenjem hardverskih resursa i primenom novih tehnika omogućili ovakvu obradu kao na primer OpenPose opisan u prethodnom poglavlju.

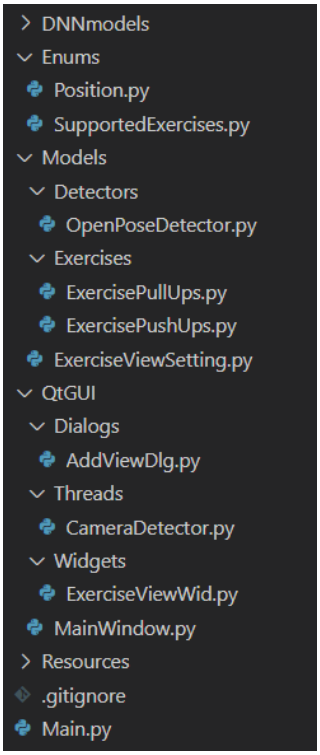
Detekcija poze na videu u realnom vremenu otvara mogućnost primene u različitim oblastima, a jedna od njih je sport. U konkretnoj realizaciji projekta, kreirana je aplikacija koja se može koristiti kao asistencija u vežbanju. Ideja aplikacija je da korisnik može imati nadgledan tening kod kuće ili na nekom drugom mestu bez potrebe asistencije personalnog trenera. Imajući u vidu da postoji veliki broj vezbi, gde većina može biti veoma kompleksna, treba napomenuti da kreirani projekat prikazuje samo koncept implementacije na primeru zgibova. Kod je napisan u programskom jeziku Python. Za kreiranje grafičkog interfejsa korišćen je Qt frejmvork, odnosno njegova ekstenzija za Python *pyqt5*. Za detekciju poze na osnovu videa korišćena je biblioteka otvorenog koda OpenPose, koja daje visoku preciznost u realnom vremenu. Kao pomoćna biblioteka za čitanje frejma sa ulazne kamere i kasniju njegovu dodatnu obradu korišćena je biblioteka OpenCV koja se koristi u oblasti računarskog vida. Pored toga OpenCV je neophodan i za rad same OpenPose biblioteke. I jedna i druga biblioteka su inicijalno napisane u C++ programskom jeziku, ali imaj svoja proširenja za Python zbog njegove popularne primene u oblasti računarskog vida.

Projekat je kreiran kao desktop aplikacija sa modularnom strukturom i sa mogućnošću proširenja funkcionalnosti. Cela struktura projekta prikazana je na slici 38. Modul QtGUI je zadužen za rad grafičkog korisničkog interfejsa i oslanja se na Qt frejmvork. Kod Qt-a se koristi koncept Widget-a (klasa QWidget slika 37) koji je osnovna gradivna komponenta koja može da odgovara na nastale događaje koristeći mehanizam signala i slotova. Signal se emituje od strane neke komponente kada nastane određeni događaj. Widget može imati više predefinisanih slotova, gde slot predstavlja funkciju koja se poziva kao odgovor na određeni signal. Widget sadrži i razne druge opcije koje se koriste za opis prozora odnosno widget-a kao što su pozicija i veličina, pozicija kursora, i tako dalje. Glavna klasa QWidget je izvedena iz osnovne klase Qt frejmvorka QObject i koristi sistem roditelja, odnosno jedan widget u sebi može sadržati druge (decu) widget-e kojima je on roditelj i upravlja njima. Widget takođe može biti i zaseban prozor ukoliko nema roditelja. Glavna klasa koja predstavlja glavni prozor aplikacije je QMainWindow u čijem se konstruktoru instanciraju svi ostali widget-i. Sledeća bitna komponenta koja se koristi jeste QDialog, tj. pomoćni prozor koji se otvara kako bi omogućio dodatnu interakciju sa korisnikom, kao na primer zahtevao određeni unos ili u konkretnom primeru zahtevao odabir ulazne kamere i odabir vežbe koja se prati.



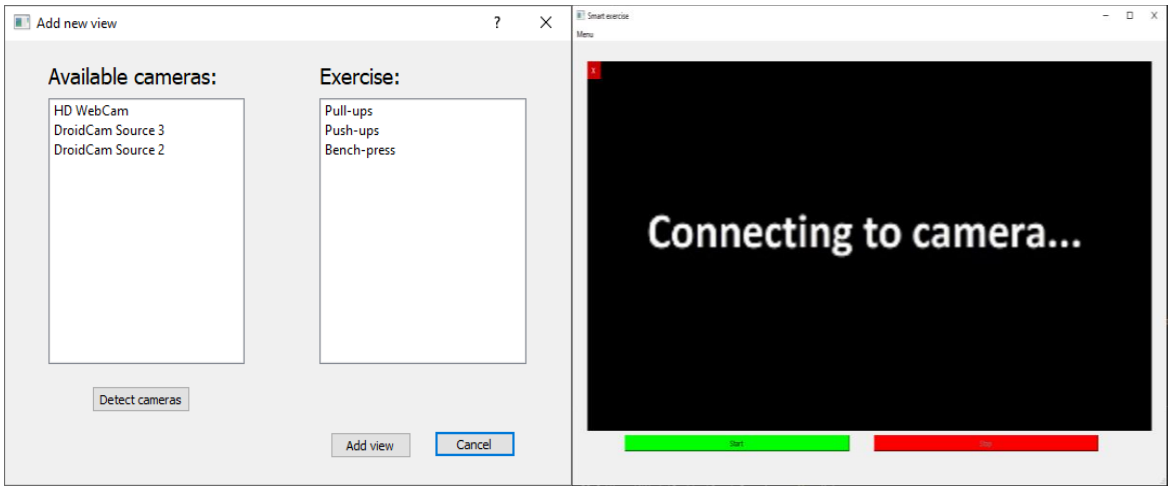
Slika 31: Qt hijerarhija klasa [20]

QMainWindow izvršava glavnu nit aplikacije koje ne treba biti blokirana izvršenjem dugih operacija pa se za te operacije koriste takozvani radnici (eng. *workers)* koji predstavljaju zasebne niti izvršenja. Klase koje predstavljaju radnika su izvedene iz Qt klase QThread i obavljaju zahtevne ili dugotrajne akcije. Sam Qt frejmvork daje podršku višenitnom okruženju, pa je celokupan postupak oko kreiranja i pokretanja niti dosta pojednostavljen. U konkretnom primeru projekta u modulu QtGUI nalaze se klase koje čine celokupan korisnički interfejs aplikacije i sastoji se iz klase *MainWindow* koja je izvedena iz QMainWindow i predstavlja glavni prozor, zatim widget-a *ExerciseViewWid* koja je izvedena iz klase QWidget i sadži sve neophodne komponente za prikaz i obradu ulaznog videa i klase *AddViewDlg* koja je izvedena iz klase QDialog i predstavlja dijalog koji se koristi za odabir ulazne kamere i odabir vežbe. Pored ovih vizuelnih komponenti u okviru ovog modula nalazi se i jedna klasa radnik *CameraDetector* izvedena iz QThread i zadužena je za obradu ulaznih video frejmova u zasebnoj niti omogućavajući na taj način nesmetan rad glavne niti. Glavna skripta *Main.py* pokreće aplikaciju i otvara glavni prozor.



Slika 32: Struktura projekta

Izgled dijaloga za odabir ulazne kamere i odabir željene vežbe prikazan je na slici 39 levo. Nakon odabira željenih parametara dijalog ih uz pomoć pomoćne klase *ExerciseViewSetting* prosleđuje glavnoj klasi *MainWindow* koja na osnovu tih parametara instancira widget *ExerciseViewWid* i prikazuje ga unutar glavnog prozora (slika 39 desno).



Slika 33: Dijalog za dodavanje opcija za rad aplikacije (levo) i widget za prikaz i obradu ulaznih frejmova (desno)

Na slici 40 prikazan je kod konstruktora widget klase *ExerciseViewWid* gde se vidi instanciranje klase radnika, zatim povezivanje signala *output* iz klase radnika sa slotom *updateFrame*, instanciranje UI komponenti unutar widget-a i zatim pokretanje niti radnika. Ovakva veza signala i slota omogućava obradu frejmova nezavisnu od njihovih prikaza. Radna nit će nakon obrade frejma, obrađeni frejm preko signala poslati widget-u koji će samo ažurirati piksel mapu svoje labele *lblFrame*.

```
def __init__(self, viewSetting, ID, parent=None):
    QtWidgets.QWidget.__init__(self, parent)

    self.myID = ID
    self.myThread = CameraDetector(viewSetting)
    self.myThread.output.connect(self.updateFrame)

    self.setupUi()

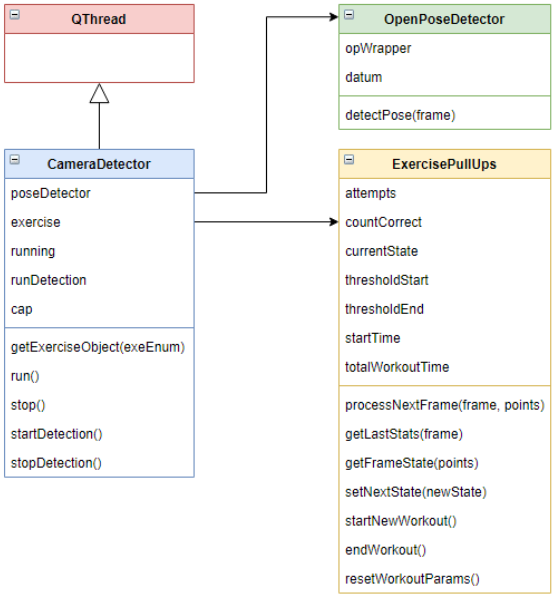
    self.myThread.start()

@pyqtSlot(QPixmap)
def updateFrame(self, pix):
    self.lblFrame.setPixmap(pix)
```

Slika 34: Konstruktor klase ExerciesViewWid i slot za prikaz obrađenog frejma

Widget klasa pored labele za prikaz frejma u sebi sadrži i dva dugmeta (tipa QPushButton) za kontrolu toka obrade frejma pozivajući odgovarajuće funkcije radnika. Nakon povezivanja sa odabranom kamerom korisnik pritiskom na dugme *start* pokreće detekciju poze tela i započinje izvođenje odabrane vežbe. Slično tome, pritisak na dugme *stop* zaustavlja detekciju tela kada je korisnik završio izvođenje vežbe i tada mu se prikazuje statistika njegovog vežbanja. Ovakva struktura omogućava laku proširljivost i nezavisan razvoj funkcionalnosti, na taj način što odvaja komponentu za prikaz od komponente koja implementira funkcionalnost.

Na sledećem klasnom dijagramu (dijagram 1) prikazana je arhitektura klasa zaduženih za obradu ulaznih frejmova za konkretan primer praćenja rada zgibova. Klasa *CameraDetektor* je kao što je ranije napomenuto izvedena iz Qt klase QThread i predstavlja klasu radnika. Zadatak ove klase je otvaranje video toka i čitanje frejmova sa odabrane kamere, obrada frejma, praćenje vremena obrade i izračunavanje broja frejmova po sekundi (FPS), konvertovanje frejma u sliku kompatibilnu sa Qt frejmvorkom i emitovanje slike preko signala *output*. Za obradu jednog frejma klasa koristi jednu instancu klase detektora poze i jednu instancu klase zadužene za praćenje izvođenja vežbe. U konkretnom slučaju koristi se instanca klase *OpenPoseDetector* koja koristi OpenPose biblioteku opisanu u prethodnom poglavlju i klasa *ExercisePullUps* koja na osnovu detektovanih ključnih tačaka (zglobova) vrši praćenje izvođenja vežbe zgibovi. Ovakva struktura klasa takođe omogućava laku proširljivost i modifikaciju. Jednostavnim promenama instanci detektora i određene vežbe može se upotrebiti drugi tip detektora i drugi tip vežbe.



Dijagram 1: Dijagram klasa zaduženih za obradu frejma

Na slici 41 prikazan je kod konstruktora klase *CameraDetektor* u kome se na osnovu prosleđenih podešavanja definišu sve neophodne promenljive. Na osnovu enumeracije iz podešavanja za izabranu vežbu instancira se objekat klase odgovarajuće vežbe. Zatim se kreira objekat klase *OpenPoseDetector* i otvara video tok za kameru sa identifikatorom proseleđenim kroz podešavanja (*setting*) uz pomoć OpenCV funkcije *VideoCapture*. Takođe definisan je i signal *output* iznad konstruktora tako da se njegovim emitovanjem šalje Qt mapa piksela, tj. QPixmap.

```
class CameraDetector(QThread):

    output = pyqtSignal(QPixmap)

    def __init__(self, setting, parent=None):
        QThread.__init__(self, parent)
        self.exercise = self.getExerciseObject(setting.getExerciseEnum())
        self.poseDetector = OpenPoseDetector()
        self.running = True
        self.runDetection = False
        self.cap = cv2.VideoCapture(setting.getCamId(), cv2.CAP_DSHOW)
```

Slika 35: Konstruktor klase CameraDetector

Pokretanjem niti radnika *CameraDetector* zapravo se pokreće prepisana (*override*) funkcija *run* iz QThread. Funkcija *run* (slika 42) predstavlja glavnu funkciju izvršenja ove niti. Na početku se pokušava čitanje frejma sa već otvorenog video toka preko promenljive *cap* i ukoliko postoji ulaz i dok lokalna promenljiva *running* ima vrednost *True,* odnosno dok traje izvršenje niti, vrši se obrada frejma. Nakon pritiska na dugme *start* lokalna promenljiva *runDetection* se postavlja na vrednost *True* i tada kreće postupak detekcije ključnih tačaka i njihovo praćenje. Pročitani frejm se prosleđuje detektoru kako bi se na njemu detektovala poza tela pozivom funkcije *detectPose* i očekuje se obrađeni frejm (nacrtan sleket ili originalni frejm) i lista *points* koja se sastoji iz 25 elemenata, gde svaki element predstavlja lokaciju određene ključne tačake na telu kao lista x i y koordinata. Nakon detekcije poze dobijeni frejm i lista ključnih tačaka se prosleđuju objektu određene vežbe na dalju obradu pozivom funkcije *processNextFrame.* Implementacija ove dve ključne funkcije biće opisana u nastavku rada. Pošto se izvrši obrada jednog frejma, sledi izračunavanje parametra frejma po sekundi i on se zajedno sa ukupnim vremenom ispisuje na obrađeni frejm pozivom OpenCV funkcije *putText*, čime je obrada frejma završena. Potom sledi konvertovanje frejma u QPixmap i emitovanje dobijene slike preko signala *output* nakon čega se on prikazuje korisniku.

```
def run(self):
    hasFrame, frame = self.cap.read()
    previousTime = time.time()

    if hasFrame:
        while self.running:
            hasFrame, frame = self.cap.read()

            if self.runDetection:
                frame, points = self.poseDetector.detectPose(frame)
                frame = self.exercise.processNextFrame(frame, points)
            else:
                frame = self.exercise.getLastStats(frame)

            currentTime = time.time()
            timeTaken = currentTime - previousTime
            fps = 1 / timeTaken
            previousTime = currentTime

            cv2.putText(frame, "time taken = {:.2f} sec ({} FPS)"
                    .format(timeTaken, int(fps)), (25, 25), cv2.FONT_HERSHEY_COMPLEX,
                    .8, (255, 50, 0), 2, lineType=cv2.LINE_AA)

            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            h, w, c = frame.shape
            step = c * w
            qImg = QImage(frame.data, w, h, step, QImage.Format_RGB888)

            self.output.emit(QPixmap.fromImage(qImg))
            cv2.waitKey(1)

    self.cap.release()
```

Slika 36: Glavna funkcija klase CameraDetector

Kao što je rečeno, klasa *OpenPoseDetector* je zadužena za detekciju ključnih tačaka vežbača na pročitanom frejmu i za detekciju koristi biblioteku OpenPose. Za korišćenje API-ja biblioteke koristi se wrapper napisan za Python, jer je biblioteka inicijalno pisana na C++ programskom jeziku. Za korišćenje biblioteke neophodno je izvršenje sledeće sekvence akcija (slika 43), koje se nalaze unutar konstruktora klase *OpenPoseDetector*.

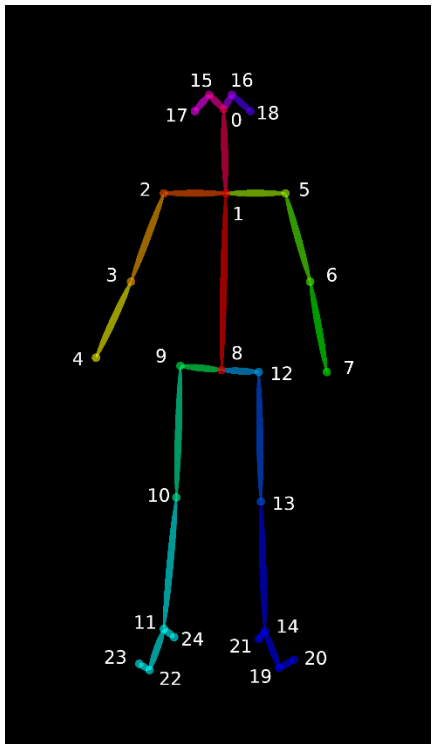
```
params = dict()
params["model_folder"] = "DNNmodels"

self.opWrapper = op.WrapperPython()
self.opWrapper.configure(params)
self.opWrapper.start()

self.datum = op.Datum()
```

Slika 37: Pokretanje biblioteke OpenPose u Python-u

Rad biblioteke zahteva korišćene unapred istreniranih modela konvolucionih neuronskih mreža koji dolaze zajedno sa bibliotekom. U prethodnom poglavlju je opisano da postoje 3 modela mreža koje su trenirane nad različitim skupovima podataka (*MPII*, *COCO*, *COCO+foot*). Model *COCO+foot* se takođe naziva i *BODY_25,* zbog toga što se kao izlaz dobija 25 ključnih tačaka čovekovog tela (slika 44), tako da će u nastavku rada biti korišćen taj naziv. U opisu rada je prikazano i poređenje u performansama između ova tri modela, gde *BODY_25* daje najveću brzinu sa korišćenjem CUDA i ujedno veliku preciznost. Upravo je to ono što je potrebno za rad jedne ovakve aplikacije. Brzina je veoma bitan faktor zbog rada u realnom vremenu, dok je preciznost važna za precizno praćenje izvođenja vežbe. Pored navedenog, *BODY_25* model je zgodan za praćenje poze vežbača jer postoje dodatne tačke u predelu karlice (tačka 8, slika 44), vrha grudnog koša (tačka 1, slika 44) i na stopalima. Tačke 1 i 8 svojim vezama omogućavaju posmatranje dva odvojena skeleta, gornjeg i donjeg dela tela, dok tačke na stopalima mogu pomoći recimo u paralelnom držanju raskoračnog stava ili pravilnoj poziciji skočnog zgloba. Klasa određene vežbe koja prati pozu vežbača pratiće samo ključne tačke bitne za izvođenje te vrste vežbe.



Slika 38: Ključne tačke modela BODY_25 [3]

Parametri rada biblioteke se podešavaju pozivom *wrapper* funkcije *configure* kojoj se prosleđuju preko Python *dict* (*dictionary*) strukture podataka. U konkretnom primeru kao parametar je prosleđena putanja do modela istreniranih mreža, dok ostali parametri imaju predefinisanu (*default*) vrednost. Predefinisana vrednost za korišćenje modela *model_pose* je "BODY_25", za broj GPU-a koji se koriste *num_gpu* je -1 što znači da koristi sve dostupne GPU-ove na mašini, za maksimalnu brzinu procesiranja frejma *fps_max* je -1 što znači najbrže moguće, za veličinu ulazne slike u mrežu *net_resolution* je "-1x368", -1 znači da će se automatski izabrati najbolji odnos veličina u zavisnosti od ulaznog frejma, a za izlaznu rezoluciju *output_resolution* je "-1x-1" što znači da će rezolucija izlazne slike (frejma) biti identična ulaznoj.

Wrapper funkcija *start* na slici 43 je zadužena za pokretanje više niti unutar same biblioteke, s tim da se ne blokira nit koja je poziva. Nakon poziva ove funkcije, biblioteka je spremna za prijem ulaznih podataka (frejmova) i detekciju poze na njima. Instanca klase *Datum* na slici 43 se koristi za prenos podataka ka i iz biblioteke, a takođe se koristi i unutar same biblioteke kao osnovna struktura za prenos podataka i informacija između niti. Glavna funkcija unutar klase *OpenPoseDetektor* je *detectPose* (slika 45) koja od niti radnika dobija pročitani frejm na obradu. Pošto se instanca klase *Datum* koristi za prenos podataka ka biblioteci prvo je potrebno smestiti ulazni frejm u ovaj objekat, odnosno njegov atribut *cvInputData* koji prihvata originalnu ulaznu sliku dimenzija *(širina_frejma x visina_frejma) x 3 kanala.* Nakon smeštanja frejma, poziva se wrapper funkcija *emplaceAndPop* kojoj se prosleđuje objekat klase *Datum* i koja vrši smeštanje podataka u red za obradu, zatim čeka da obrada bude završena i čita obrađene podatke iz reda.

```python
def detectPose(self, frame):
    self.datum.cvInputData = frame
    self.opWrapper.emplaceAndPop([self.datum])

    points = []
    if self.datum.poseKeypoints.ndim == 3:
        for i in range(25):
            x = int(self.datum.poseKeypoints[0, i, 0])
            y = int(self.datum.poseKeypoints[0, i, 1])

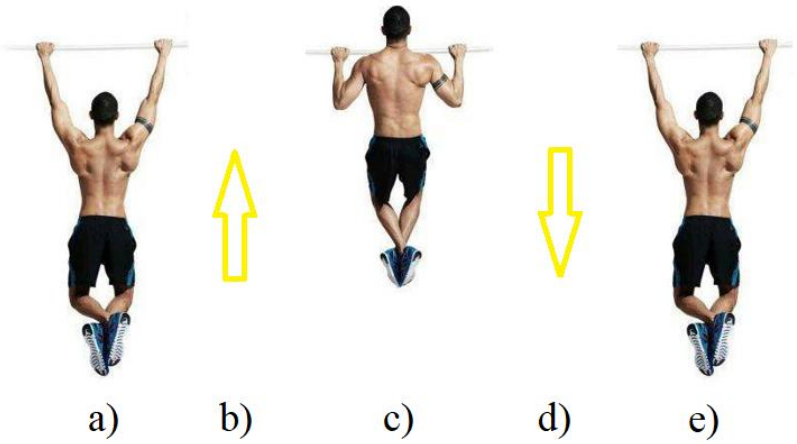
            if x == 0 and y == 0:
                points.append(None)
            else:
                points.append((x, y))
    else:
        points = [None] * 25

    return self.datum.cvOutputData, points
```

Slika 39: Glavna funkcija klase OpenPoseDetector

Nakon obrade podaci se nalaze unutar *datum* objekta u njegovim atributima *cvOutputData* i *poseKeypoints.* Atribut *cvOutputData* čuva obrađen frejm (originalni frejm + nacrtan skelet preko tela) veličine (*izlazna_visina x izlazna_širina) x 3 kanala*, a rezolucija je definisana flegom *output_resolution* koji je opisan u prethodnom pasusu. U atributu *poseKeypoints* se čuvaju predviđene lokacije ključnih tačaka, u originalu je to C++ niz (*Array*) veličine *broj_ljudi x delovi_tela x 3*, gde se broj 3 odnosi na (x, y koordinate + verovatnoća), a u Python-u se originalan niz predstavlja preko niza tipa *numpy* (*ndarray*). Treba napomenuti da su koordinate skalirane u odnosu na rezoluciju izlaznog frejma. Zatim se u petlji *numpy* niz ključnih tačaka prebacuje u listu *points* u formatu koji se koristi za njihovu dalju obradu, gde element liste sadrži samo (x, y) koordinate, a redni broj elementa je redni broj ključne tačke na slici 44. Nakon obrade niza, kao izlaz iz funkcije se vraćaju obrađeni frejm i lista ključnih tačaka.

Posle završene detekcije ključnih tačaka, prihvaćeni frejm i lista ključnih tačaka se prosleđuju funkciji *processNextFrame* objekta klase *ExercisePullUps* koja na osnovu njih vrši praćenje izvođenja vežbe. Pre nego sto bude opisana sama implementacija praćenja, potrebno je definisati pravila na osnovu kojih će biti određeno da je vežba pravilno izvedena ili ne. U konkretnom slučaju kod zgibova može se definisati pravilno odrađen zgib sledećom sekvencom: pravilna početna pozicija (slika 46a), pravilno podizanje do krajnje tačke (slika 46c) i pravilno vraćanje u početnu poziciju (slika 46e). Na osnovu ovih pravila mogu se definisati određena stanja (pozicije).



Slika 40: Pravilan zgib

Ukoliko obuhvatimo sve slučajeve, mogu se izdvojiti sledeća stanja, prikazana na slici 47:

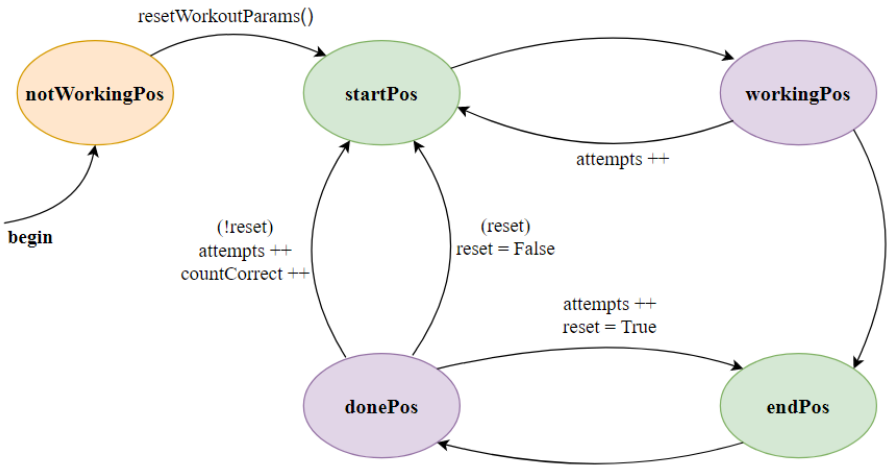
- **notWorkingPos** – definiše poziciju u kojem se vežbač ne drži za šipku (ne vežba)
- **startPos** – (slika 46a) definiše poziciju u kojoj je vežbač drži za šipku u pravilnom početnom položaju (obe ruke ispružene ili blago savijene u laktovima)
- **workingPos** – (slika 46b) definiše stanje u kojem vežbač podiže telo od startne ka krajnjoj poziciji
- **endPos** – (slika 46c) definiše poziciju u kojoj se vežbač podigao do krajnje tačke
- **donePos** – (slika 46d) definiše stanje u kojem se vežbač vraća iz krajnje u startnu poziciju.

```
class Position(Enum):
    notWorkingPos = 0
    startPos = 1
    workingPos = 2
    endPos = 3
    donePos = 4
```

Slika 41: Sva moguća stanja pri izvođenju zgiba (enumeracija)

Pošto su definisana sva moguća stanja, definišu se parametri na osnovu kojih se može kasnije izvesti statistika. U konkretnoj implementaciji (dijagram 1 klasa žutom bojom) koriste se dve promenljive *attempts* koja predstavlja ukupan broj pokušaja i promenljiva *countCorrect* koja predstavlja broj korektno odrađenih zgibova. Takođe kao pomoćni fleg se koristi promenljiva *reset.*

Nakon definisanja svih mogućih stanja i parametara za statistiku, praćenje izvođenja zgiba može se predstaviti mašinom stanja i akcijama koje se dešavaju pri prelazu iz jednog u drugo stanje. Početno stanje je *notWorkingPos* iz kog se jedino može preći u *startPos* i tom prilikom se parametri statistike postavljaju na 0. Iz stanja *startPos* jedino sledeće stanje je *workingPos,* odnosno prelazi se kada vežbač krene da se podiže. Kada se vežbač nalazi u stanju *workingPos* moguća su dva scenarija, ili će se vratiti u početno stanje pre nego što se podigao do kraja ili će se podići do krajnje pozicije. U prvom slučaju smatra se da je vežbač neuspešno odradio zgib. Nakon što korisnik dodje u krajnju poziciju (*endPos*) on može preći u *donePos,* odnosno stanje kada se korisnik spušta ka početnoj poziciji. Kada se vežbač nalazi u *donePos,* takođe su moguća dva scenarija. Ili će se vežbač vratiti u početni položaj čime se smatra da je uspešno odradio zgib i tom prilikom mu se povećava broj pokušaja i broj uspešno odrađenih zgibova ili će se podići do krajnje pozicije pre nego što se spusti do početne pozicije što se podrazumeva kao nepravilno izveden zgib i broj pokušaja mu se povećava. **Napomena:** kod scenarija gde se korisnik nađe u sekvenci stanja *endPos – donePos – endPos – donePos – startPos* to se računa kao jedan pokušaj i u tu svrhu se koristi pomoćni fleg *reset.* Ukoliko je fleg *True* znači da se vežbaču već uračunao jedan pokušaj i tada se fleg samo vraća na vrednost *False,* dok u situaciji kada je fleg *False* zapravo imamo pravilnu promenu stanja.



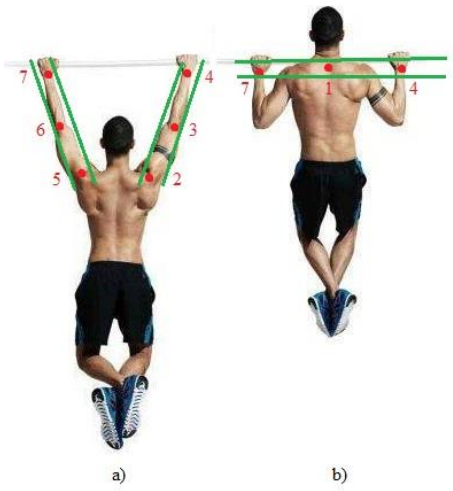
Dijagram 2: Sva moguća stanja i akcije pri prelazima

Sledeći problem koji je potrebno rešiti je kako na osnovu detektovanih ključnih tačaka odrediti stanje u kojem se vežbač nalazi. Na početku treba definisati koje su ključne tačke važne za određenu vežbu. U konkretnom primeru zgibova potrebno je pratiti ključne tačke na rukama (rame, lakat, zglob šake) i ključnu tačku vrha grudnog koša. Ukoliko ključne tačke dobijemo iz modela *BODY_25* (slika 44) onda su tačke za praćenje (slika 48):

- desna ruka – rame (tačka 2), lakat (tačka 3), zglob šake (tačka 4)
- leva ruka – rame (tačka 5), lakat (tačka 6), zglob šake (tačka 7)
- vrh grudnog koša – tačka 1.

Nakon sagledavanja ovih tačaka u početnoj poziciji (*startPos* – slika 48a) i krajnjoj poziciji (*endPos* – slika 48b), a znajući da detektovane lokacije tačke predstavljaju relativne x i y koordinate u odnosu na frejm, za detekciju te dve pozicije može se iskoristiti jednačina prave kroz dve tačke:

$$y - y1 = \frac{y2 - y1}{x2 - x1}(x - x1).$$



Slika 42: Ključne tačke bitne za praćenje rada zgiba

Međutim ukoliko posmatramo osnovni oblik jednačine prave, to znači da će prava prolaziti striktno kroz sve tri tačke, na primer kroz tačke 5, 6 i 7, što u konkretnoj primeni nije neophodno. Zato će biti korišćena mala modifikacija. Nakon sređivanja osnovnog oblika i uvođenjem promenljive *z* jednačina ima oblik:

$$z = \frac{y2 - y1}{x2 - x1}(x - x1) - y + y1, \quad (5)$$

gde *z* predstavlja određeno odstupanje, za $z = 0$ sve tri tačke pripadaju istoj pravoj. Nakon modifikovanja jednačine može se uvesti prag *p* (eng. *threshold*) na osnovu koga će se računati da li tri tačke zadovoljavaju uslov za određenu poziciju preko uslova:

$$|z| \leq p\,.$$

Vizuelno je prag predstavljen zelenom bojom na slici 48a i 48b. Praktično to znači da za početnu poziciju ruke mogu biti potpuno opružene ili blago savijene. Još jedna dobit uvođenja praga jeste što i pored visoke preciznosti detektora može doći do određenih fluktuacija pozicija ključnih tačaka koje mogu izazvati pogrešna detektovanja pozicija vežbača. U konkretnoj implementaciji klasa *ExercisePullUps* koristi dva praga, jedan za *startPos* (*thresholdStart*) i jedan za *endPos* (*thresholdEnd*), sa vrednostima 30.0 i 15.0 respektivno.

Na slici 49 je prikazana glavna funkcija klase *ExercisePullUps* zadužena za procesiranje frejma i ključnih tačaka dobijenih obradom *OpenPoseDetector-a*. Funkciju poziva nit radnik odmah nakon završene obrade i dobijanja rezultata detektora ključnih tačaka.

```
def processNextFrame(self, frame, points):
    newState = self.getFrameState(points)
    self.setNextState(newState)

    self.totalWorkoutTime = time.time() - self.startTime
    cv2.putText(frame, self.currentState.name, (25, 50), cv2.FONT_HERSHEY_COMPLEX, .8,
                (0, 0, 0), 2, lineType=cv2.LINE_AA)
    self.showStat(frame)

    return frame
```

Slika 43: Glavna funkcija za procesiranje frejma u klasi ExercisePullUps

Unutar funkcije *processNextFrame* (slika 49) prvo se na osnovu prosleđenih ključnih tačaka određuje nova pozicija vežbača pozivom funkcije *getFrameState* čija implementacija se nalazi na slici 50. Funkcija kao argument prima kompletnu listu lokacija svih 25 ključnih tačaka, a kao izlaz generiše poziciju vežbača na tom frejmu i vraća je preko enumeracije *Position.* U funkciji se izdvajaju tačke od značaja za pravilno izvođenje zgiba i preko jednačine (5) se računaju *Zsp* i *Zep* koji predstavljaju odstupanje za *startPos* i *endPos* respektivno. Nakon toga se primenjuje poređenje sa pragovima i na osnovu njih određuje novo stanje vežbača.

```
def getFrameState(self, points):
    # right hand
    point2 = points[2]
    point3 = points[3]
    point4 = points[4]

    if point2 is not None and point3 is not None and point4 is not None:
        if point4[1] < point2[1]:
            if point2[0] != point4[0]:
                Zsp = ((point2[1] - point4[1]) / (point2[0] - point4[0])) * (point3[0] - point4[0]) - point3[1] + point4[1]
            else:
                Zsp = point3[0] - point2[0]

            if abs(Zsp) <= self.thresholdStart:
                return Position.startPos

    # left hand
    point5 = points[5]
    point6 = points[6]
    point7 = points[7]

    if point5 is not None and point6 is not None and point7 is not None:
        if point7[1] < point5[1]:
            if point5[0] != point7[0]:
                Zsp = ((point5[1] - point7[1]) / (point5[0] - point7[0])) * (point6[0] - point7[0]) - point6[1] + point7[1]
            else:
                Zsp = point6[0] - point5[0]

            if abs(Zsp) <= self.thresholdStart:
                return Position.startPos

    # endPos
    pointEnd = points[1]
    if pointEnd is not None and point7 is not None and point4 is not None:
        if point7[0] != point4[0]:
            Zep = ((point7[1] - point4[1]) / (point7[0] - point4[0])) * (pointEnd[0] - point4[0]) - pointEnd[1] + point4[1]
        else:
            Zep = point4[1] - pointEnd[1]

        if abs(Zep) <= self.thresholdEnd:
            return Position.endPos
        elif Zep > 0:
            return Position.endPos

    return Position.workingPos
```

Slika 44: Implementacija funkcije za određivanje nove pozicije vežbača

Posle određivanja nove pozicije vežbača poziva se funkcija *setNextState* koja na osnovu prethodne pozicije i trenutne (nove) vrši promenu stanja vežbača, odnosno implementira mašinu stanja. Implementacija te funkcije prikazana je na slici 51. Nakon promene stanja preračunava se ukupno vreme vežbanja i ispisuje na frejm pozivom OpenCV funkcije *putText.*

```
def setNextState(self, newState):
    if self.currentState is Position.notWorkingPos:
        if newState is Position.startPos:
            self.currentState = Position.startPos
            self.resetWorkoutParams()
    else:
        if newState is Position.notWorkingPos:
            self.currentState = Position.notWorkingPos
            self.endWorkout()

        elif self.currentState is Position.startPos:
            if newState is Position.workingPos:
                self.currentState = Position.workingPos

        elif self.currentState is Position.workingPos:
            if newState is Position.startPos:
                self.currentState = Position.startPos
                self.attempts += 1
            elif newState is Position.endPos:
                self.currentState = Position.endPos

        elif self.currentState is Position.endPos:
            if newState is Position.workingPos:
                self.currentState = Position.donePos

    elif self.currentState is Position.donePos:
        if newState is Position.startPos:
            self.currentState = Position.startPos
            if self.reset:
                self.reset = False
            else:
                self.attempts += 1
                self.countCorrect += 1
        elif newState is Position.endPos:
            self.currentState = Position.endPos
            self.attempts += 1
            self.reset = True
```

Slika 45: Funkcija zadužena za promenu stanja u kom se nalazi vežbač

Zaključak

Problem detekcije ljudske poze je odavno poznat u računarkom vidu i predmet je istraživanja proteklih tridesetak godina. Predstavljeno je puno radova i implementacija na ovu temu, ali najveći uspeh do tog trenutka je postiglo korišćenje dubinskih kamera od pre desetak godina. Zatim su krenula istraživanja da li je dubinska kamera odnosno dubinska slika zaista neophodna za detekciju poze ili je to moguće korišćenjem obične RGB slike. Ideja o korišćenju veštačkih neuronskih mreža u računarstvu poznata je nauci dugi niz godina, ali su one svoj vrhunac dosegle tek u ovoj deceniji. Dodavanje konvolucionih slojeva arhitekturi neuronske mreže omogućilo je da mreža dobije sposobnost obrade slika. Nakon ILSVRC takmičenja 2013, 2014 i 2015. godine, gde su se konvolucione neuronske mreže pokazale odlično u rešavanju problema klasifikacije objekata na slikama, pa čak imale rezultate bolje i od čoveka, krenula je njihova masovna primena u računarskom vidu.

Nakon uspešnog rešavanja problema klasifikacije, ConvNet mreže su se uspešno iskoristile i za rešavanje problema lokalizacije ljudskih zglobova na čovekovom telu. Prvi rad koji je predstavio rešenje detekcije poze korišćenjem ConvNet-a bio je DeepPose. DeepPose je predstavio detekciju poze kao regresioni problem i kreirao niz konvolucionih neuronskih mreža kao kaskadu zasebnih regresora, gde jedan regresor zadužen za lokalizaciju ključnih tačaka na svojoj ulaznoj slici. Ukoliko se posmatra cela kaskada, prvi regresor uzima celu sliku i vrši predikciju zglobova, a nakon njega slede regresori koji dobijaju samo delove slike oko predviđenih zglobova, čime se postiže poboljšanje rezultata predikcije u svakom sledećem koraku. DeepPose je na ovaj način istraživanja preusmerio ka metodama dubokog učenja za rešavanje problema detekcije poze.

Pokazalo se da duboko učenje, odnosno korišćenje dubokih konvolucionih neuronskih mreža, daju bolje i preciznije rezultate u detekciji. Međutim povećanje dubine CovnNet-a sa sobom nosi i veliko povećanje broja operacija koje je potrebno izvršiti. To je rezultiralo povećanjem potrebnog vremena za detekciju poze na slici, koje može biti i par sekundi. Kod praćenja poze tela na videu ovo vreme predstavlja problem, pa su pokrenuta istraživanja u pogledu omptimizacija na samim arhitekturama mreža i boljem iskorišćenju hardvera sistema. OpenPose je rad iz 2019. godine koji je predstavio detekciju poze tela u realnom vremenu. Rad se bazira na korišćenju mapi poverenja, koje predstavljaju verovatnoću da se na određenom pikselu nalazi zadati deo tela, i polja afiniteta dela tela, koja opisuju smer i orijentaciju posmatranog dela tela. Njihovom kombinacijom moguće je veoma uspešno i sa visokom reciznošću odrediti poze tela više ljudi na slici u realnom vremenu.

Detekcija poze u realnom vremenu omogućava praćenje poze tela tokom niza frejmova odnosno može se iskoristiti za nadgledanje aktivnosti čoveka, što je našlo primenu u brojnim aplikacijama koje se bave sportom, bezbednošću, animacijom... U ovom radu je predstavljen koncept korišćenja praćenja poze tela u aplikaciji koja može pomoći za praćenje i asistenciju u vežbanju. Aplikacija je razvijena na programskom jeziku Python uz pomoć Qt frejmvorka i biblioteka OpenCV i OpenPose. Qt frejmvork je korišćen za grafički korisnički interfejs i prikaz izlaznih frejmova. Kao detektor poze tela na frejmovima sa ulazne kamere korišćena je OpenPose biblioteka, koja vrši predikciju ključnih tačaka u realnom vremenu koristeći grafičke

procesore na sistemu za paralelno izvršenje velikog broja operacija. OpenCV je korišćen kao pomoćna biblioteka za dodatnu obradu frejmova. Aplikacija implementira praćenje rada zgibovova na osnovu poze vežbača, ali je zamišljena i kreirana tako da bude lako proširljiva i za druge vežbe. Potencijalna primena aplikacije može biti nadgledano praćenje izvođenja vežbe u kućnim uslovima, tj. zamena za personalnog trenera.

Reference

[1] S. C. Babu (2019), *A 2019 guide to Human Pose Estimation with Deep Learning* [Online] Dostupno na: https://nanonets.com/blog/human-pose-estimation-2d-guide/

[2] B. Raj (28.04.2019.), *An Overview of Human Pose Estimation with Deep Learning* [Online], Dostupno na: https://medium.com/beyondminds/an-overview-of-human-pose-estimation-with-deep-learning-d49eb656739b

[3] Z. Cao et al. (30.05.2019.), *OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields* [Online], Dostupno na: https://arxiv.org/pdf/1812.08008.pdf

[4] *Pose estimation guide* [Online], Dostupno na: https://www.fritz.ai/pose-estimation/

[5] D. Mwiti (08.05.2018.), *Convolutional Neural Networks: An Intro Tutorial* [Online], Dostupno na: https://heartbeat.fritz.ai/a-beginners-guide-to-convolutional-neural-networks-cnn-cf26c5ee17ed

[6] J. B. Ahire (11.11.2018.), *The Artificial Neural Networks* [Online], Dostupno na: https://medium.com/@jayeshbahire/the-artificial-neural-networks-handbook-part-4-d2087d1f583e

[7] Prabhu (04.05.2018.), *Understanding of Convolutional Neural Network (CNN) – Deep Learning* [Online], Dostupno na: https://medium.com/@RaghavPrabhu/understanding-of-convolutional-neural-network-cnn-deep-learning-99760835f148

[8] *Softmax function*, Wikipedia, [Online], Dostupno na: https://en.wikipedia.org/wiki/Softmax_function [poslednji pristup: 30. oktobar 2020.]

[9] *Convolutional Neural Networks (CNN / ConvNets)* [Online], Dostupno na: https://cs231n.github.io/convolutional-networks/

[10] Volotaire (23.05.2014.), *The motion capture in Dawn of the Planet of the Apes and new TV Spot* [Online], Dostupno na: https://www.neogaf.com/threads/the-motion-capture-in-dawn-of-the-planet-of-the-apes-and-new-tv-spot.824332/

[11] *Kinect interactive games – game development company* [Online], Dostupno na:https://x-tech.am/kinect-interactive-games-game-development-company/

[12] L. Pishchulin et al. (26.04.2016.), *DeepCut: Joint Subset Partition and Labeling for Multi Person Pose Estimation* [Online], Dostupno na: https://arxiv.org/pdf/1511.06645.pdf

[13] S. Tsang (22.08.2018.), *Review: VGGNet – 1st Runner-up (Image Classification), Winner (Localization) in ILSVRC 2014* [Online], Dostupno na: https://medium.com/coinmonks/paper-review-of-vggnet-1st-runner-up-of-ilsvlc-2014-image-classification-d02355543a11

[14] S. Tsang (24.08.2018.), *Review: GoogLeNet (Inception v1) - Winner of ILSVRC 2014 (Image Classification)*, [Online], Dostupno na: https://medium.com/coinmonks/paper-review-of-googlenet-inception-v1-winner-of-ilsvlc-2014-image-classification-c2b3565a64e7

[15] K. He et al. (10.12.2015.), *Deep Residual Learning for Image Recognition* [Online], Dostupno na: https://arxiv.org/pdf/1512.03385.pdf

[16] Y. Yang et al., *Articulated Human Detection with Flexible Mixtures-of-Parts* [Online], Dostupno na: https://www.cs.cmu.edu/~deva/papers/pose_pami.pdf

[17] A. Toshev i C. Szegedy (20.08.2014.), *DeepPose: Human Pose Estimation via Deep Neural Networks* [Online], Dostupno na: https://arxiv.org/pdf/1312.4659.pdf

[18] J. Tompson et al. (09.06.2015.), *Efficient Object Localization Using Convolutional Networks* [Online], Dostupno na: https://arxiv.org/pdf/1411.4280.pdf
[19] Z. Cao et al. (14.04.2017.), *Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields* [Online], Dostupno na: https://arxiv.org/pdf/1611.08050.pdf
[20] *Qt for Beginners* [Online], Dostupno na: https://wiki.qt.io/Qt_for_Beginners